QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Week 10 – Sem 2 – 2026

Prof Niko Suenderhauf

Welcome to Week 10!



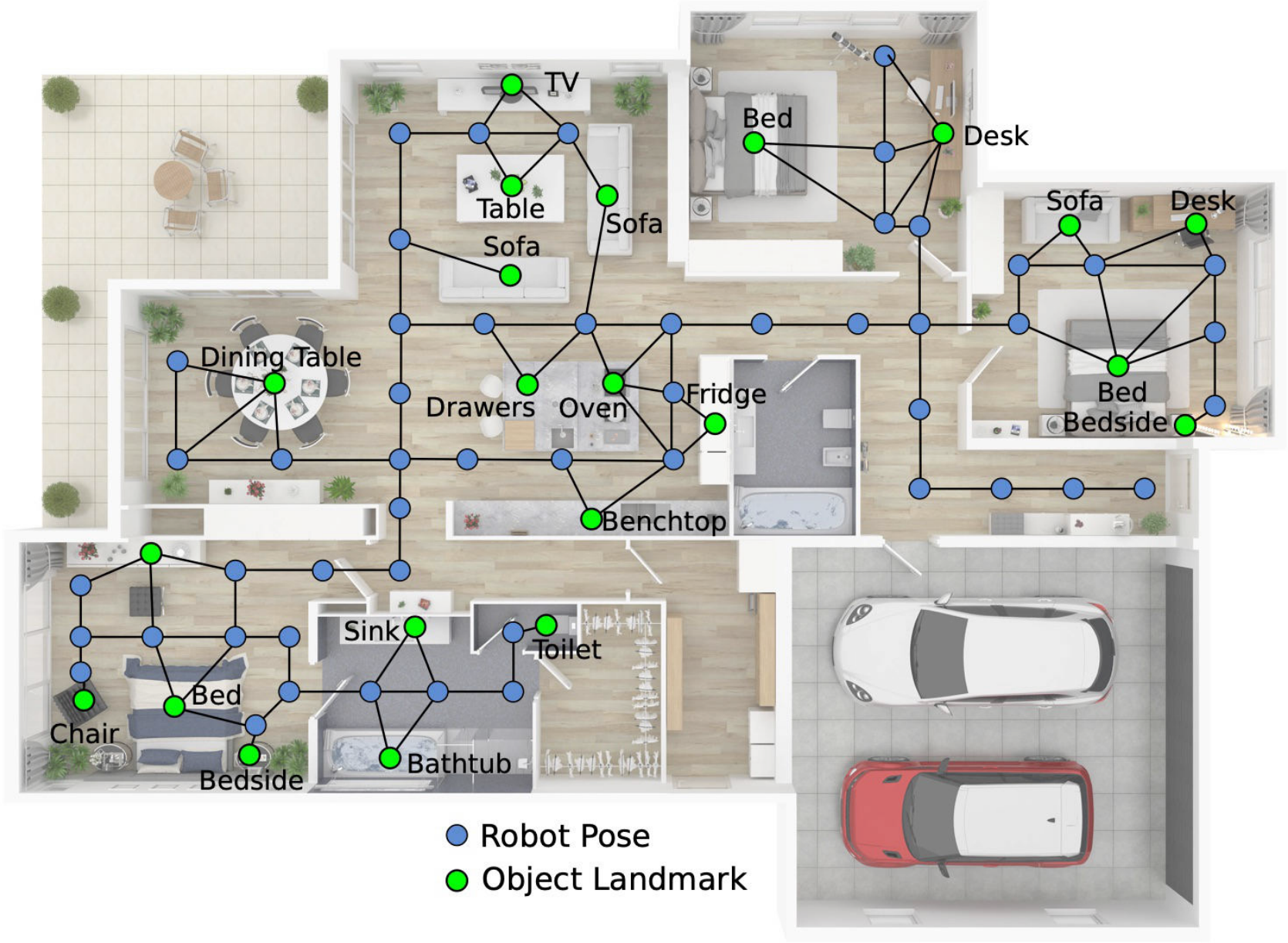
Next Week: Public Holiday on Monday!
Will focus learning on Prac & Tutorial and a recording.

CRICOS No.00213J



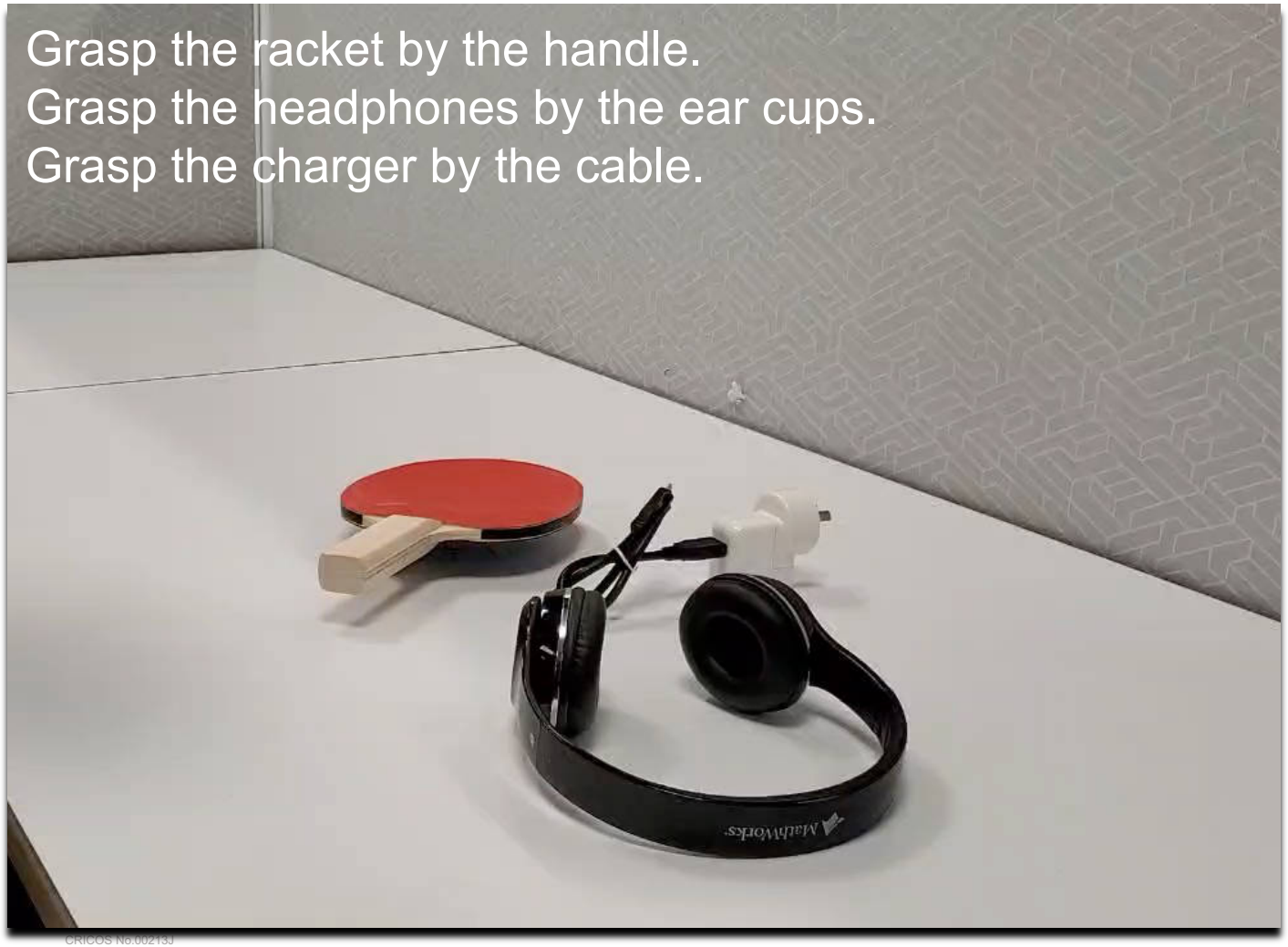


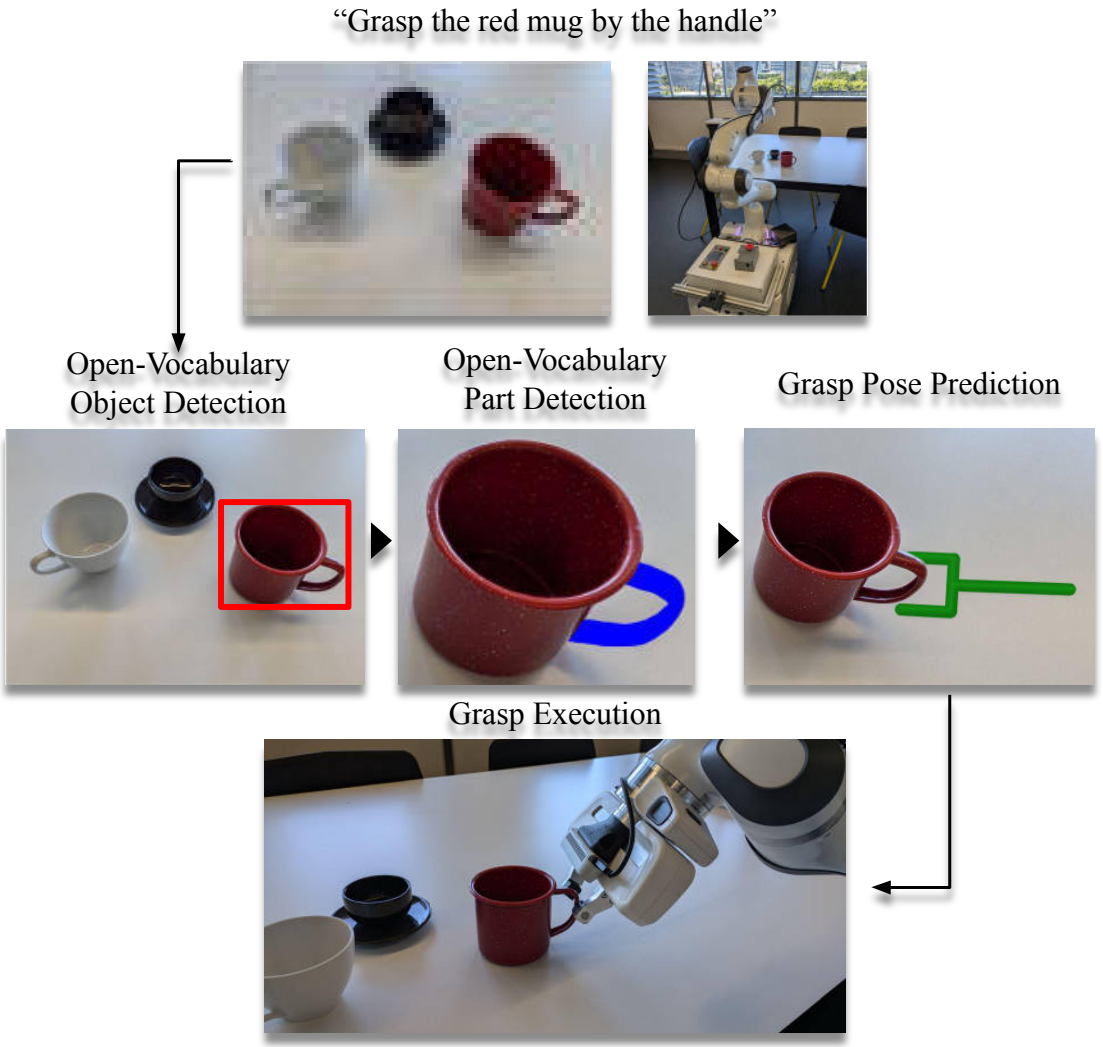
CRICOS No.00213J

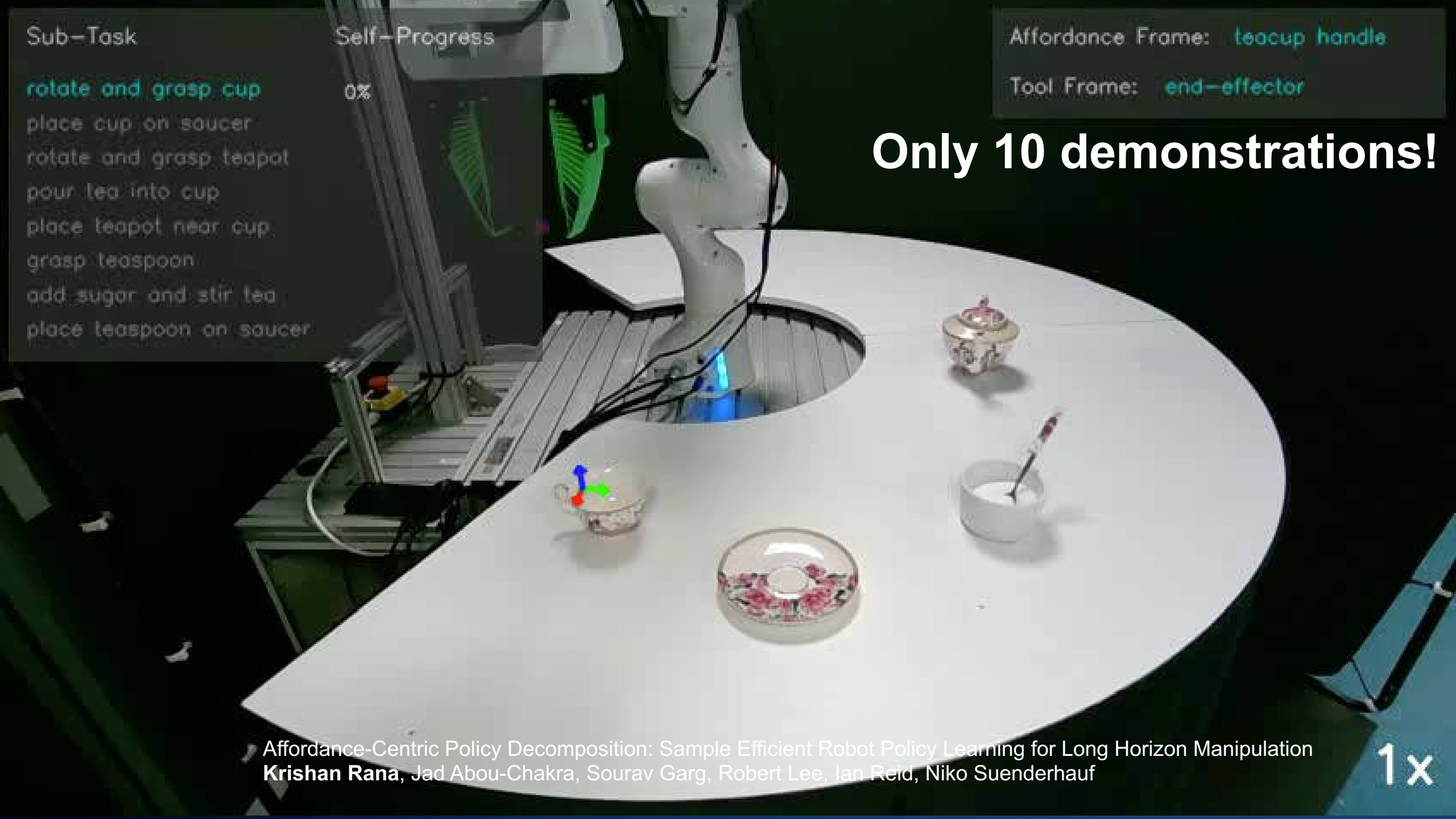



bottle|0.95
car|0.81
book|0.84
clock|0.84
cup|0.95
cup|0.96
vase|0.935
vase|0.921
keyboard|0.96
mouse|0.95
spoon|0.97
ork|0.96
knife|0.76
vase|0.864
vase|0.863
teddy bear|0.98
orange|0.88
bowl|0.97
cup|0.68
cup|0.92

Open-Vocabulary Part-Based Grasping



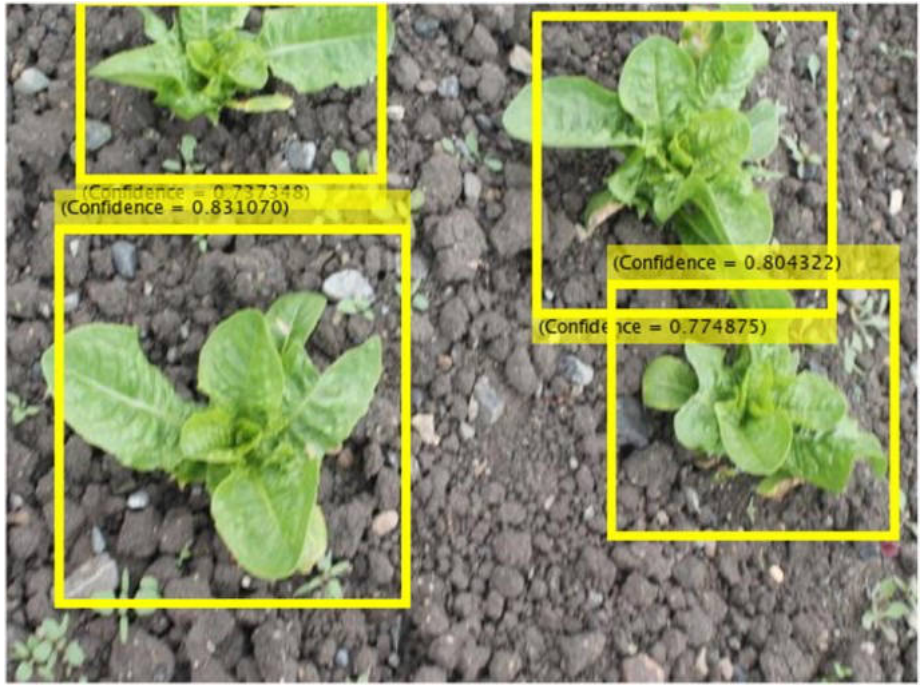




Affordance-Centric Policy Decomposition: Sample Efficient Robot Policy Learning for Long Horizon Manipulation

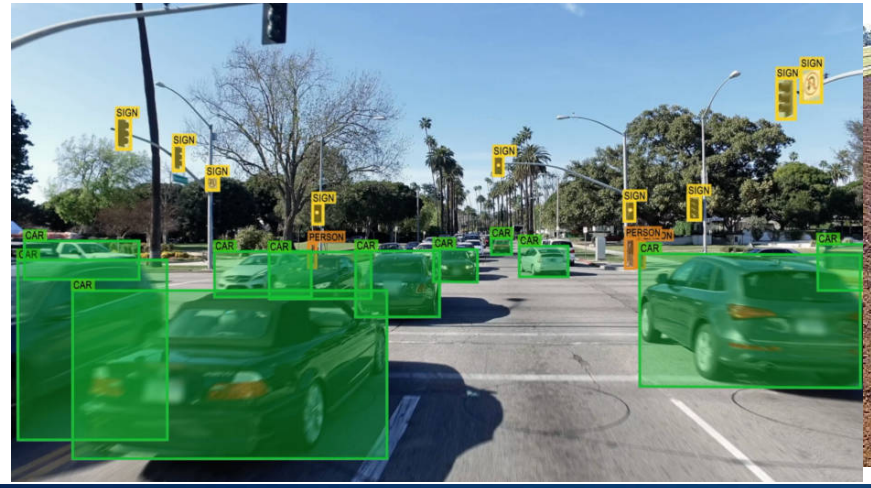
Krishan Rana, Jad Abou-Chakra, Sourav Garg, Robert Lee, Ian Reid, Niko Suenderhauf

1x





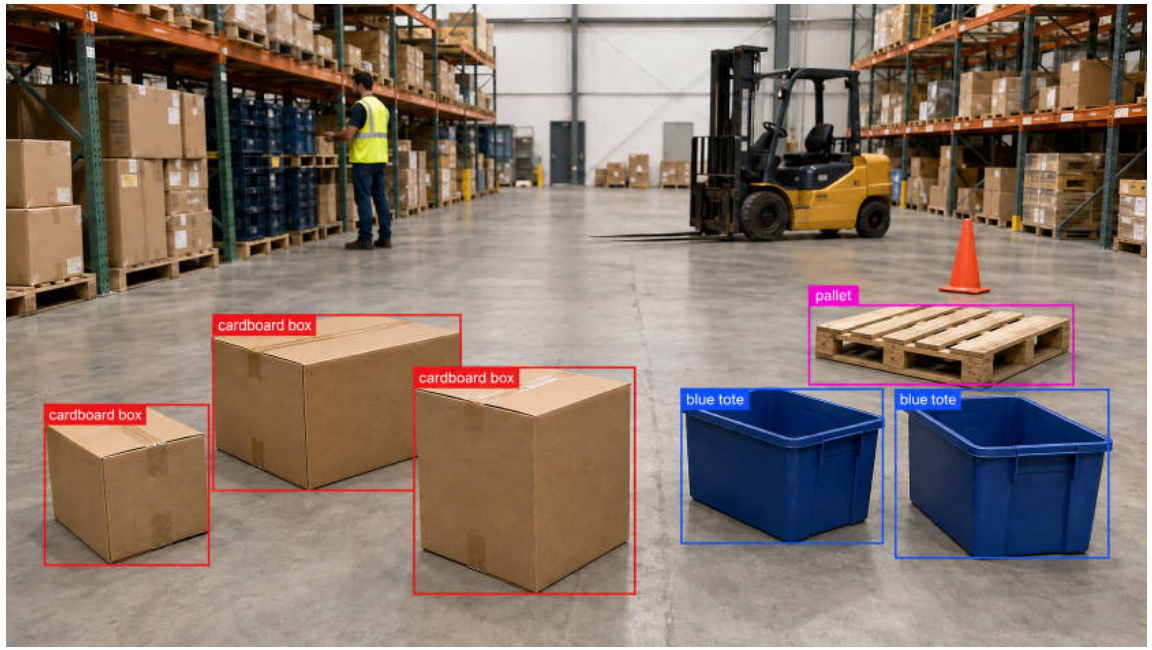






CRICOS No.00213J

Object Detection



- **Input:**
  - RGB Image
- **Output:**
  - ?
- **Supervision:**
  - ?
- **Loss (example):**
  - ?
- **Useful Metric:**
  - ?

Object Detection



- **Input:**
  - RGB Image
- **Output:**
  - A set of bounding boxes, class labels and confidence scores, one per object
- **Supervision:**
  - Bounding box and class label for each object
- **Loss (example):**
  - Classification and bounding-box regression losses
- **Useful Metric:**
  - Average precision at an IoU threshold

CRICOS No.00213J

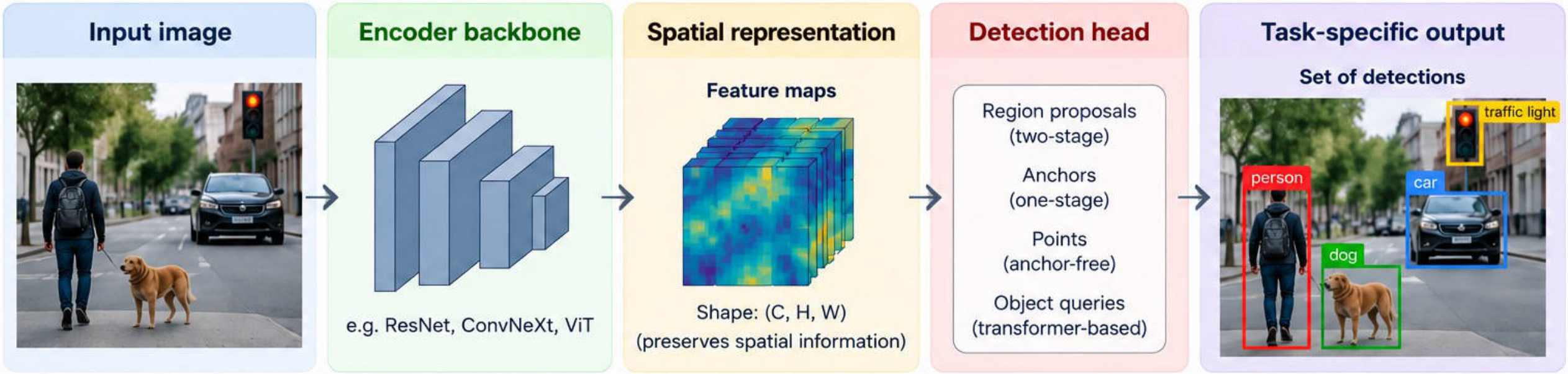
Object Detection – Output

For each detected object, output:

- **Bounding box coordinates** $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$
- **Class Label Distribution** giving a score or probability for each possible object class

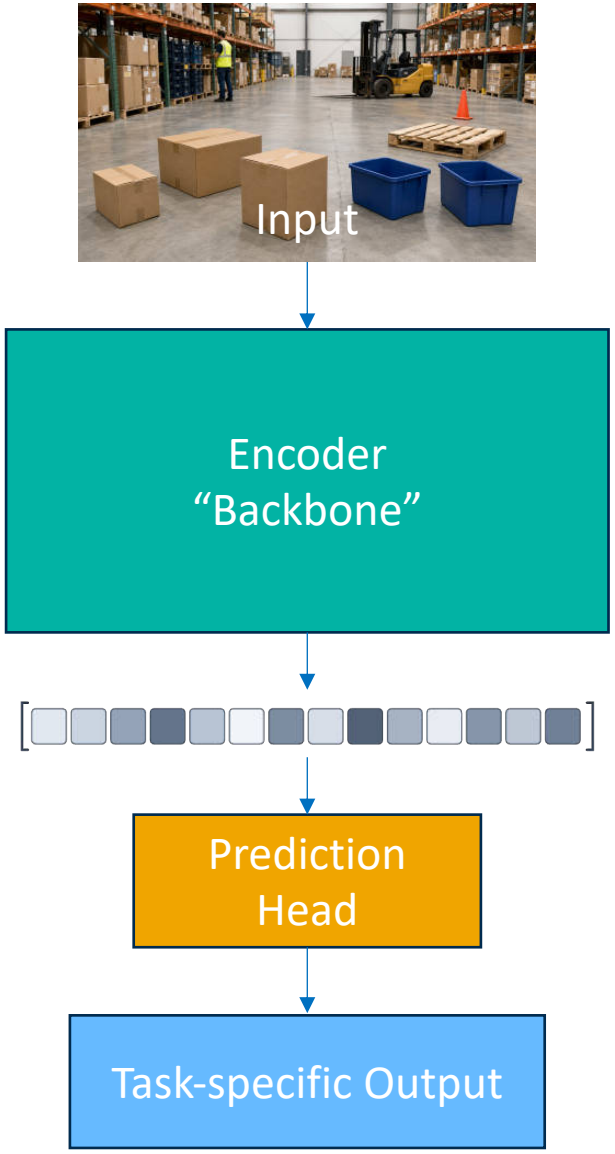


Object Detection – General Architecture



A High-Level View





What makes Object Detection architecturally more complicated than image classification or per-pixel depth prediction?

CRICOS No.00213J

The Key Challenge:
A variable Number of Objects

In image classification, segmentation, per-pixel prediction, **the output size is fixed**.

For object detection, **the number of outputs depends** on the content of the image!

How many object predictions should the detector make?

Where should those candidate object predictions come from?

CRICOS No.00213J

Candidate Objects Come From …

Different types of Object Detection solve this problem in different ways:

1. Two-stage detectors: predefined anchor boxes plus region-proposal network
2. Anchor-based one-stage detectors: predefined anchor boxes
3. Anchor-free / point-based detectors : image locations or points
4. Transformer-based detectors: learned object queries

CRICOS No.00213J

Two Stage Detectors

Two-Stage Detectors

First: find candidate object regions. Then: classify and refine them.

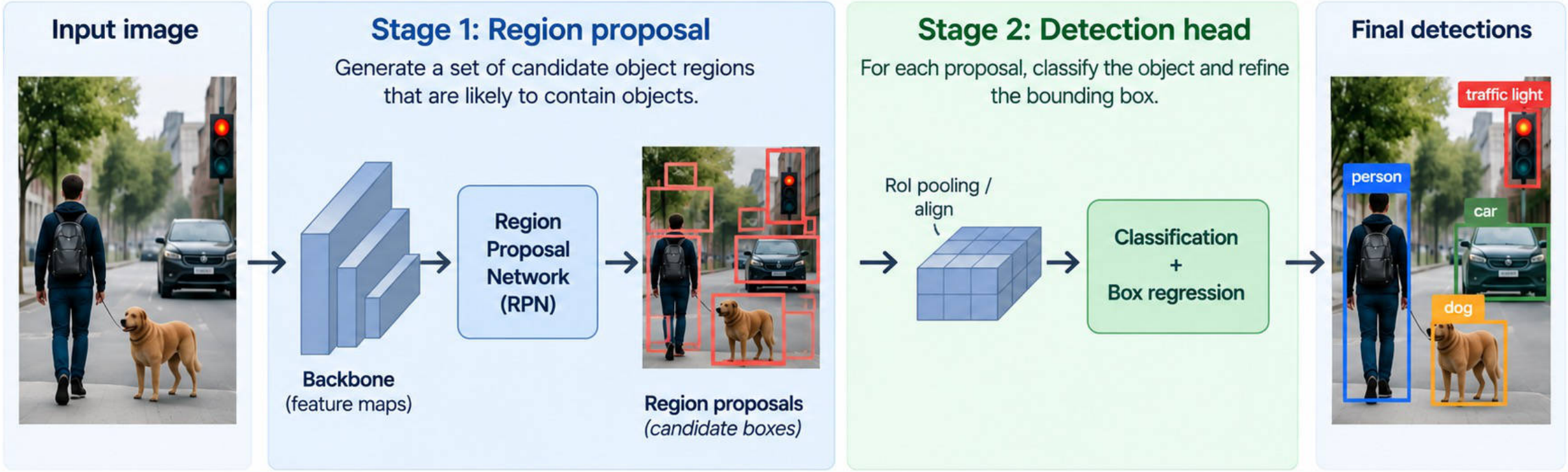
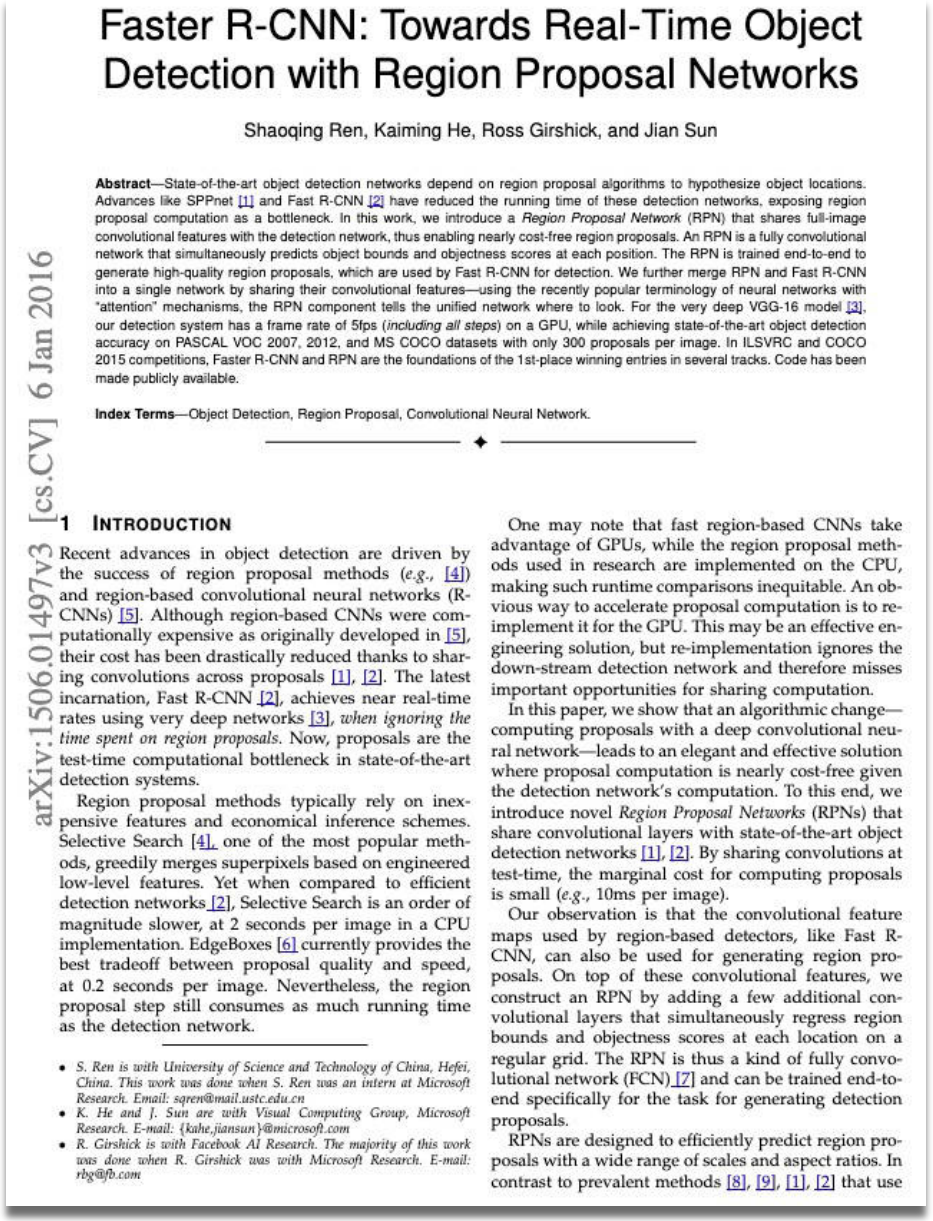


Illustration by GPT-5.6

Two-Stage Detectors
Faster R-CNN

First family of detectors developed, but still relevant today:

- Reliable **baseline** for object detection
- Modular architecture: backbone, proposal network and detection head are easy to understand and modify
- **Straightforward to fine-tune** on a custom dataset using pretrained backbones/checkpoints
- Widely implemented in major libraries such as Detectron2, MMDetection and TorchVision
- Newer descendants: Mask R-CNN (adds segments), Cascade R-CNN (progressive box refinement), Sparse R-CNN (learnable sparse proposals)



Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun

Abstract—State-of-the-art object detection networks depend on region proposal algorithms to hypothesize object locations. Advances like SPPnet [1] and Fast R-CNN [2] have reduced the running time of these detection networks, exposing region proposal computation as a bottleneck. In this work, we introduce a *Region Proposal Network* (RPN) that shares full-image convolutional features with the detection network, thus enabling nearly cost-free region proposals. An RPN is a fully convolutional network that simultaneously predicts object bounds and objectness scores at each position. The RPN is trained end-to-end to generate high-quality region proposals, which are used by Fast R-CNN for detection. We further merge RPN and Fast R-CNN into a single network by sharing their convolutional features—using the recently popular terminology of neural networks with "attention" mechanisms, the RPN component tells the unified network where to look. For the very deep VGG-16 model [3], our detection system has a frame rate of 5fps (*including all steps*) on a GPU, while achieving state-of-the-art object detection accuracy on PASCAL VOC 2007, 2012, and MS COCO datasets with only 300 proposals per image. In ILSVRC and COCO 2015 competitions, Faster R-CNN and RPN are the foundations of the 1st-place winning entries in several tracks. Code has been made publicly available.

Index Terms—Object Detection, Region Proposal, Convolutional Neural Network.

arXiv:1506.01497v3 [cs.CV] 6 Jan 2016

1 Introduction

Recent advances in object detection are driven by the success of region proposal methods (*e.g.*, [4]) and region-based convolutional neural networks (R-CNNs) [5]. Although region-based CNNs were computationally expensive as originally developed in [5], their cost has been drastically reduced thanks to sharing convolutions across proposals [1], [2]. The latest incarnation, Fast R-CNN [2], achieves near real-time rates using very deep networks [3], *when ignoring the time spent on region proposals*. Now, proposals are the test-time computational bottleneck in state-of-the-art detection systems.

Region proposal methods typically rely on inexpensive features and economical inference schemes. Selective Search [4], one of the most popular methods, greedily merges superpixels based on engineered low-level features. Yet when compared to efficient detection networks [2], Selective Search is an order of magnitude slower, at 2 seconds per image in a CPU implementation. EdgeBoxes [6] currently provides the best tradeoff between proposal quality and speed, at 0.2 seconds per image. Nevertheless, the region proposal step still consumes as much running time as the detection network.

- S. Ren is with University of Science and Technology of China, Hefei, China. This work was done when S. Ren was an intern at Microsoft Research. Email: sqren@mail.ustc.edu.cn
- K. He and J. Sun are with Visual Computing Group, Microsoft Research. E-mail: {kahe,jiansun}@microsoft.com
- R. Girshick is with Facebook AI Research. The majority of this work was done when R. Girshick was with Microsoft Research. E-mail: rbg@fb.com

One may note that fast region-based CNNs take advantage of GPUs, while the region proposal methods used in research are implemented on the CPU, making such runtime comparisons inequitable. An obvious way to accelerate proposal computation is to re-implement it for the GPU. This may be an effective engineering solution, but re-implementation ignores the down-stream detection network and therefore misses important opportunities for sharing computation.

In this paper, we show that an algorithmic change—computing proposals with a deep convolutional neural network—leads to an elegant and effective solution where proposal computation is nearly cost-free given the detection network's computation. To this end, we introduce novel *Region Proposal Networks* (RPNs) that share convolutional layers with state-of-the-art object detection networks [1], [2]. By sharing convolutions at test-time, the marginal cost for computing proposals is small (*e.g.*, 10ms per image).

Our observation is that the convolutional feature maps used by region-based detectors, like Fast R-CNN, can also be used for generating region proposals. On top of these convolutional features, we construct an RPN by adding a few additional convolutional layers that simultaneously regress region bounds and objectness scores at each location on a regular grid. The RPN is thus a kind of fully convolutional network (FCN) [7] and can be trained end-to-end specifically for the task for generating detection proposals.

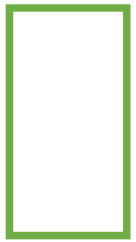
RPNs are designed to efficiently predict region proposals with a wide range of scales and aspect ratios. In contrast to prevalent methods [8], [9], [1], [2] that use

Stage 1:
Propose Initial Guesses for Object Locations

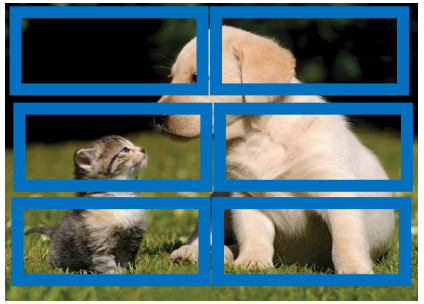
Anchor Box 1



Anchor Box 2

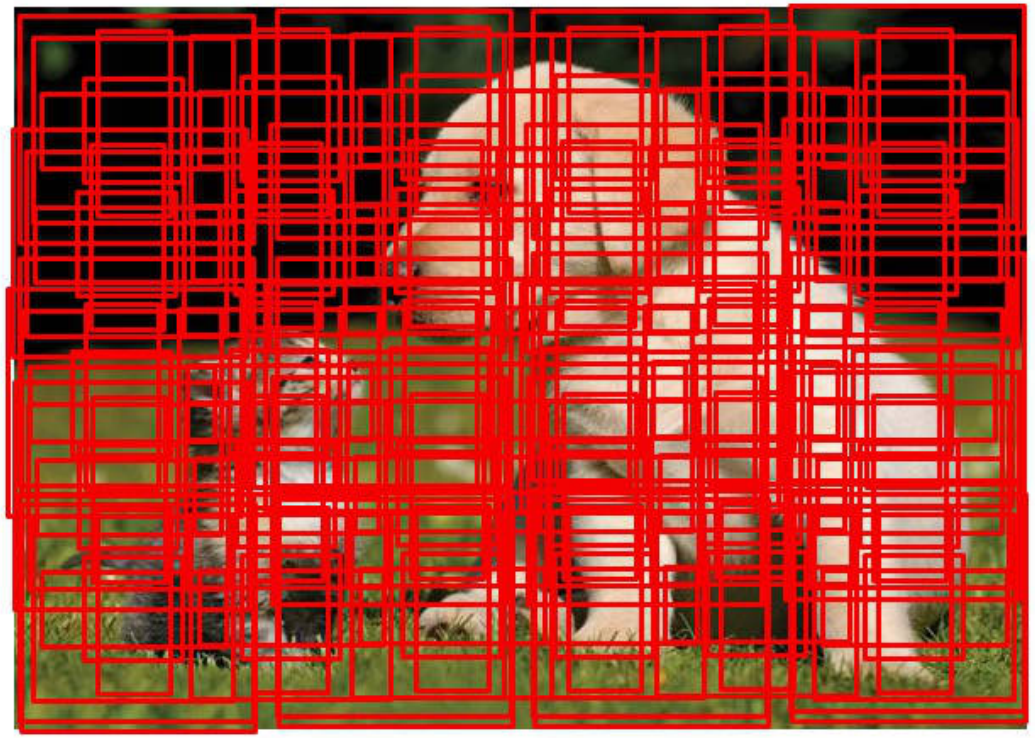


Anchor Box 1 tiled across image



Anchor Box 2 tiled across image

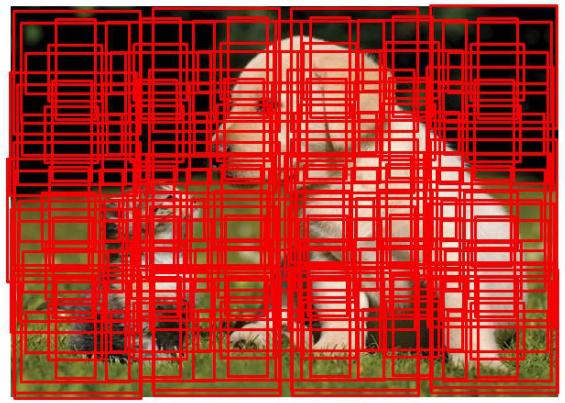


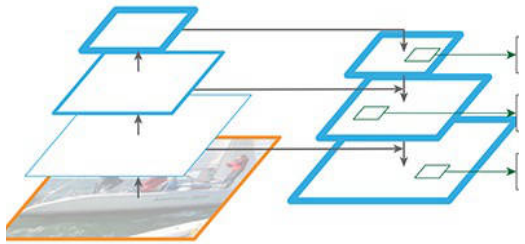


Start with a dense grid of "anchors" pre-defined by Location, Scale, Aspect ratio

CRICOS No.00213J

Stage 1:
Propose Initial Guesses for Object Locations





Region Proposal Network

1. "Objectness score" for every pre-defined box
2. Better fitting box

CRICOS No.00213J

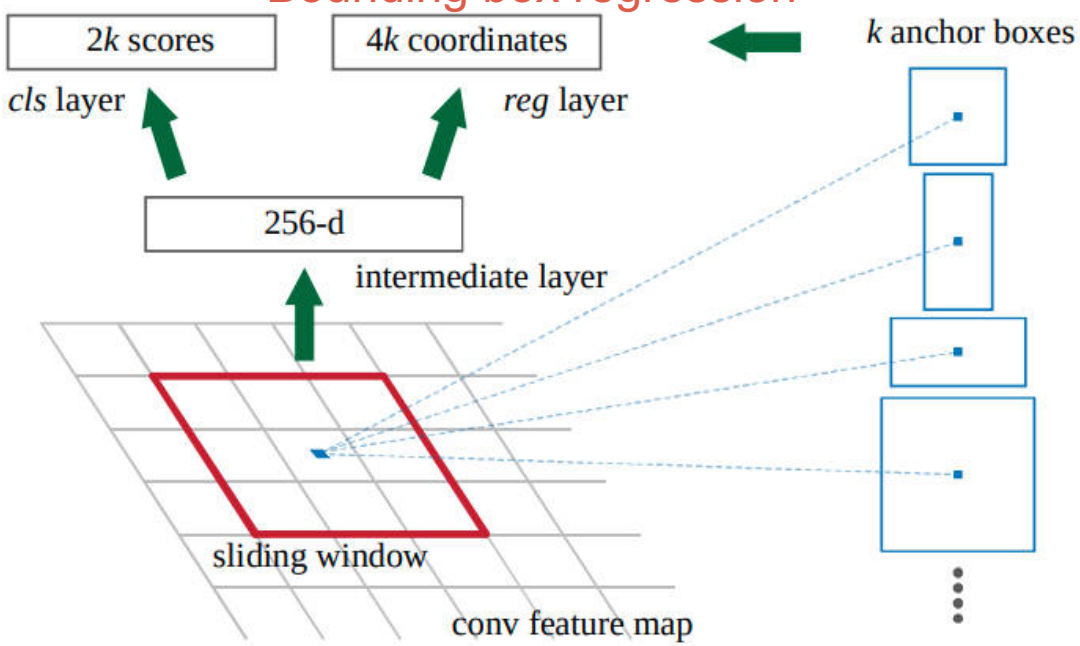
Stage 1:
Propose Initial Guesses for Object Locations

object vs. non-object classification

Bounding box regression

3 scales, 3 aspect ratios at each location of the sliding window

Region Proposal Network



CRICOS No.00213J

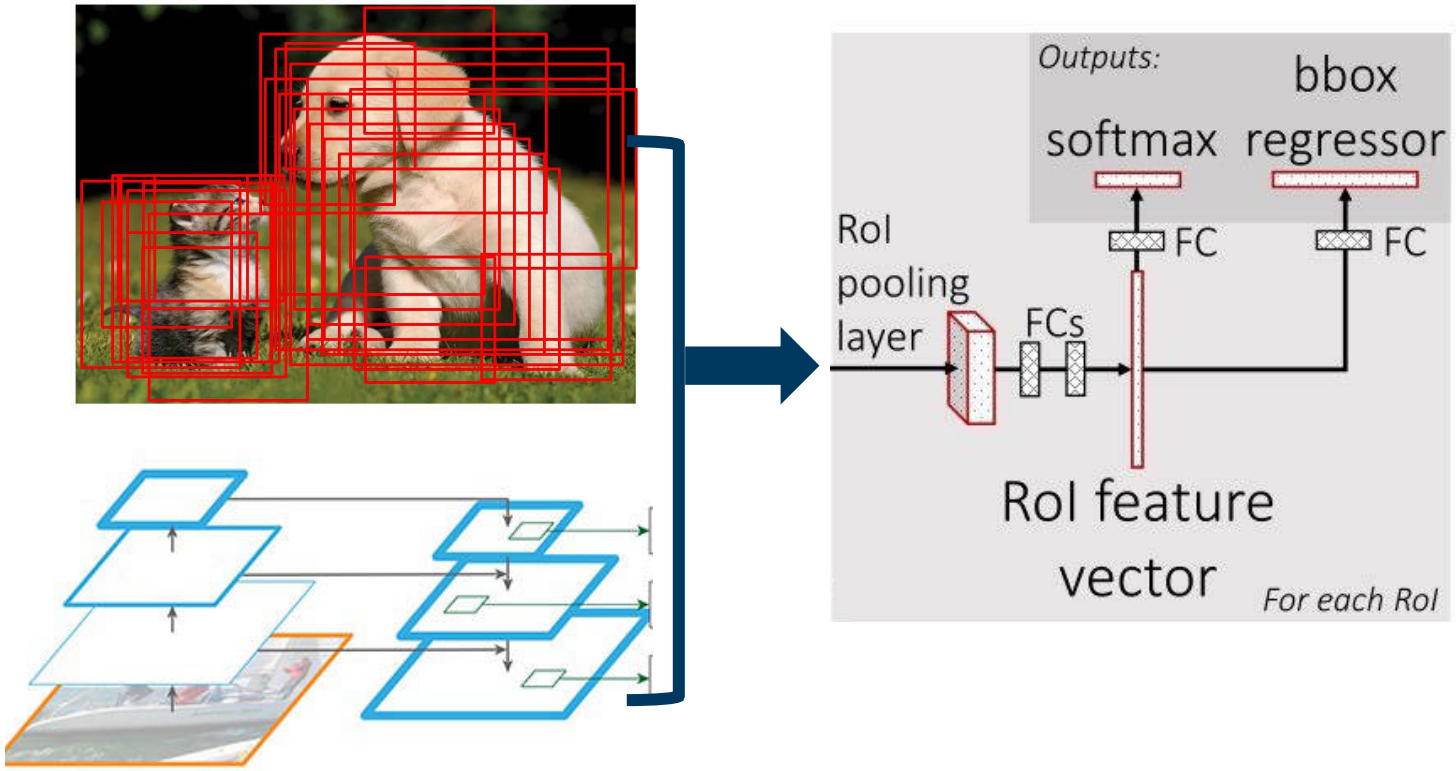
Stage 1:
Propose Initial Guesses for Object Locations

Take top-N (e.g. 1,000) region proposals based on objectness score



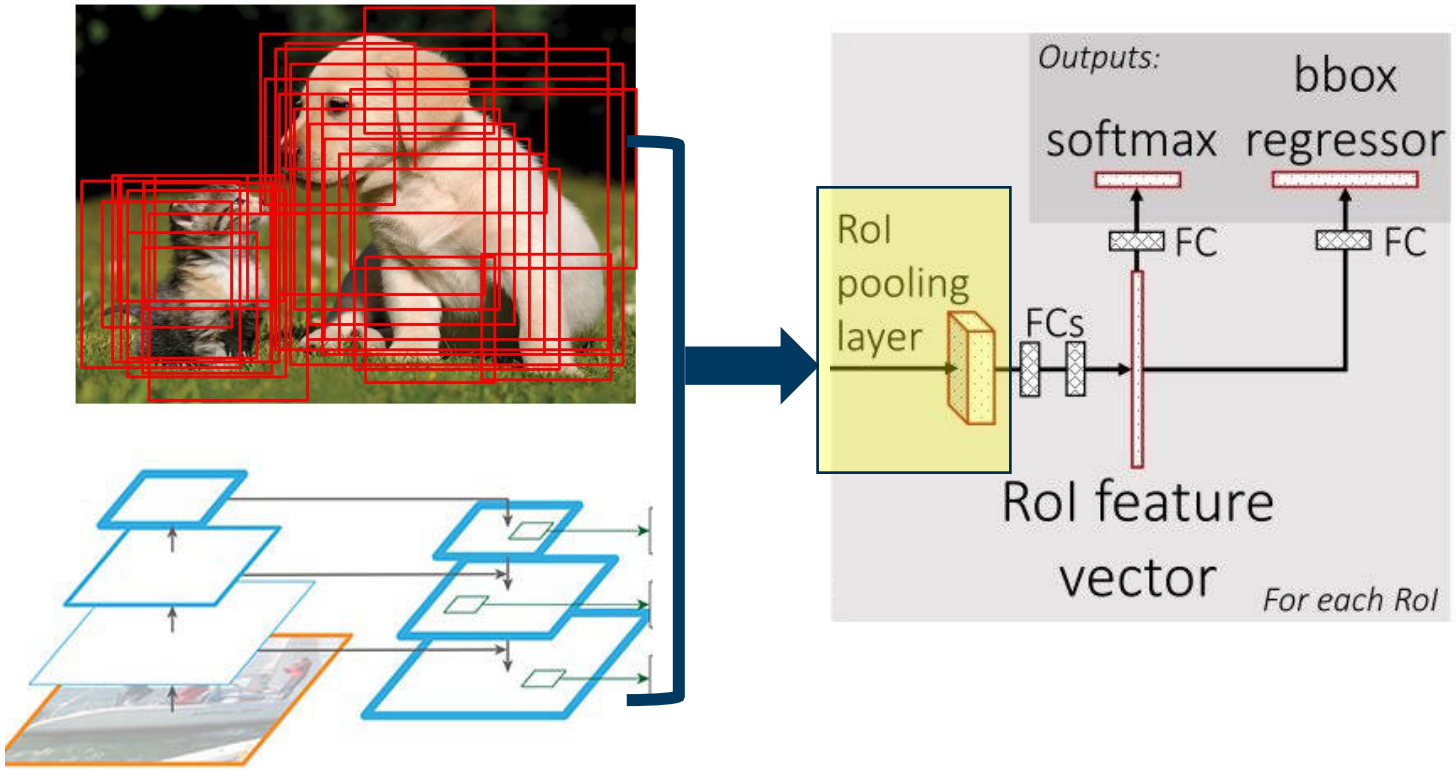
CRICOS No.00213J

Stage 2:
Refine proposals into detections



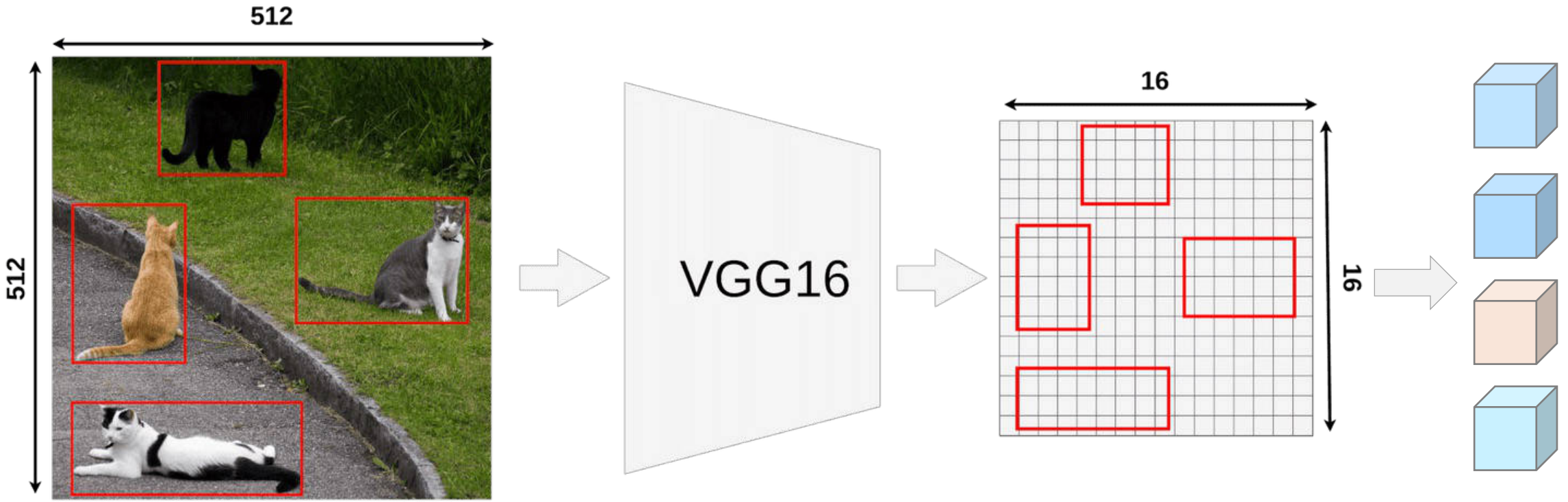
CRICOS No.00213J

Stage 2:
Refine proposals into detections



CRICOS No.00213J

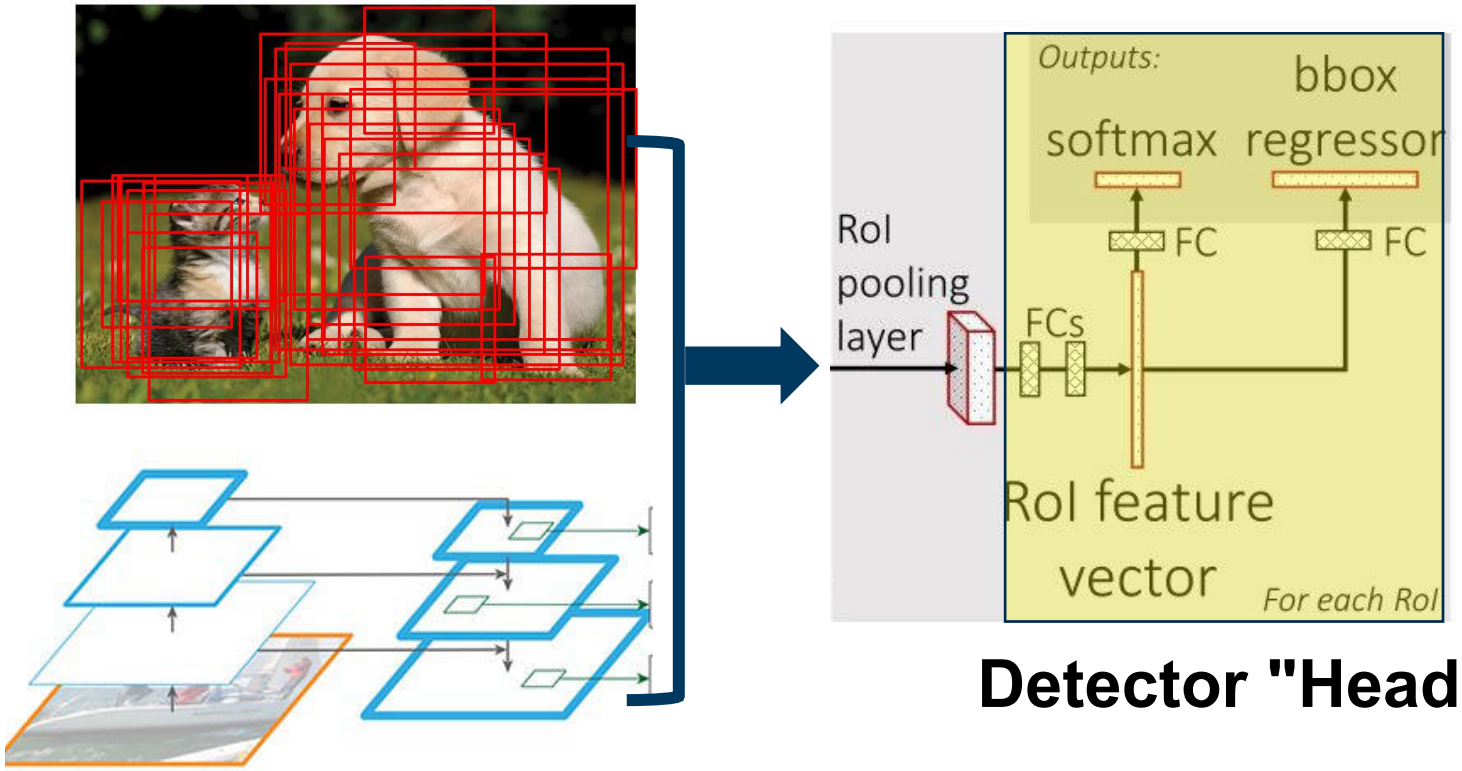
Region-of-Interest (ROI) Pooling



Pool regions of different sizes into the same size feature representation.

CRICOS No.00213J

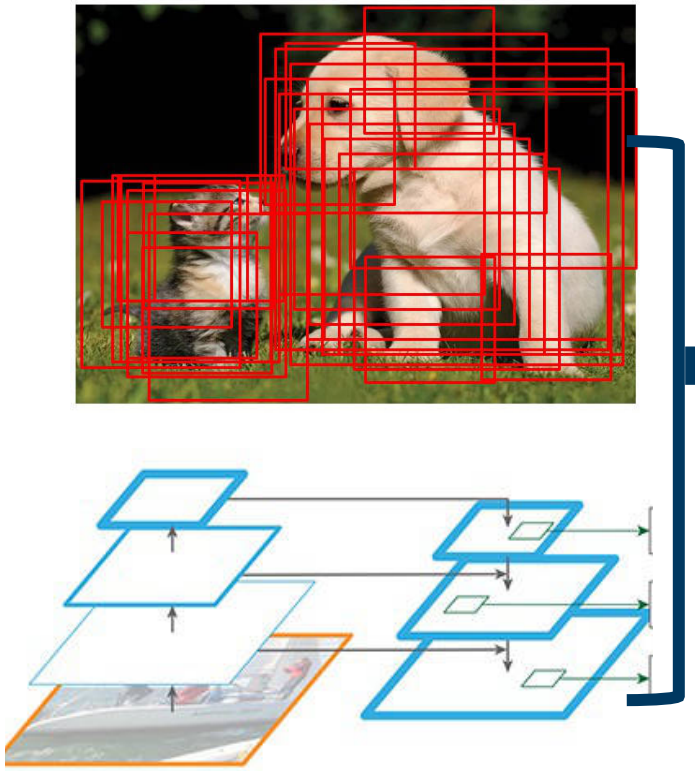
Stage 2:
Refine proposals into detections

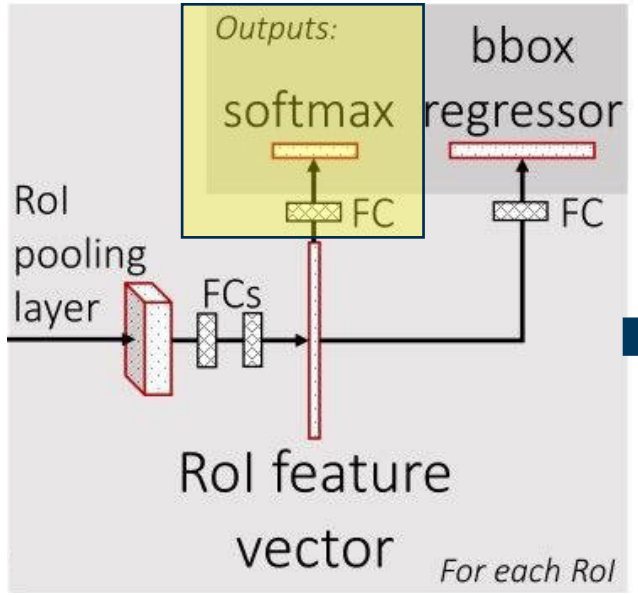


Detector "Head"

CRICOS No.00213J

Stage 2:
Refine proposals into detections





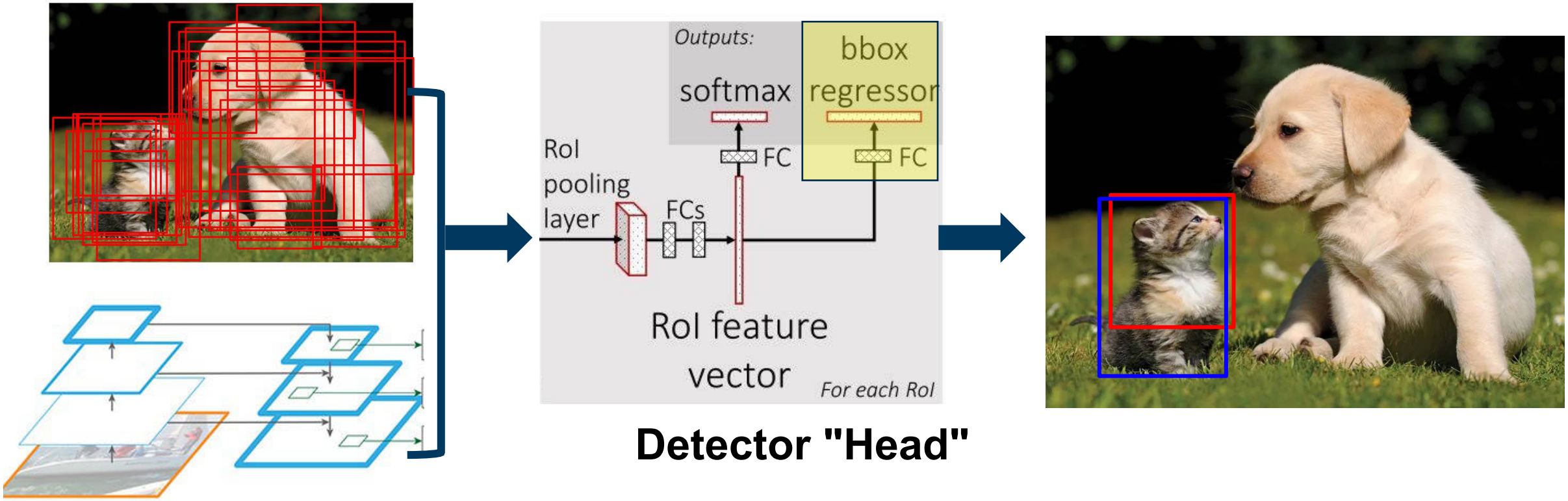
Detector "Head"

Cat: 5% Dog: 93% … Background: 2%
Cat: 87% Dog: 3% … Background: 10%
Cat: 22% Dog: 15% … Background: 63%

Cat: 0% Dog: 4% … Background: 96%

CRICOS No.00213J

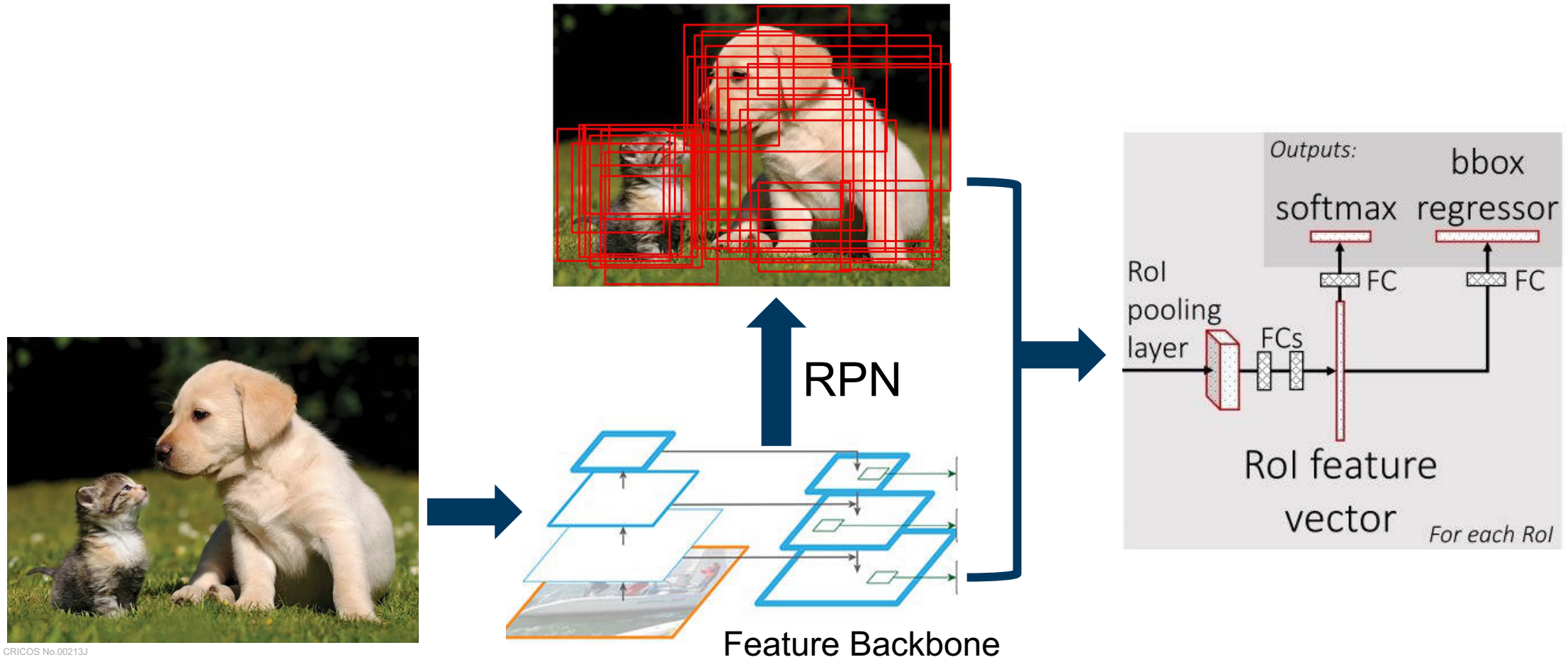
Stage 2:
Refine proposals into detections



Detector "Head"

CRICOS No.00213J

Two-Stage Object Detectors



CRICOS No.00213J

Use it yourself!

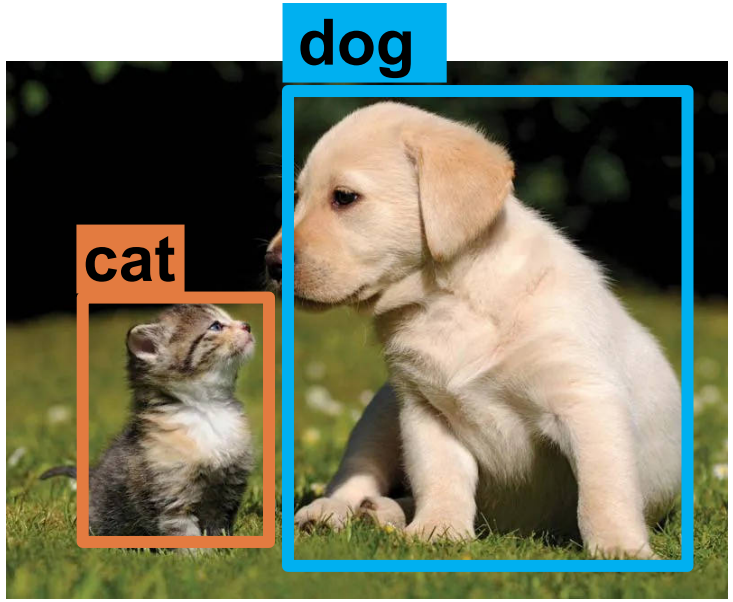
```python
import torch
from PIL import Image
from torchvision.models.detection import (
    FasterRCNN_ResNet50_FPN_Weights,
    fasterrcnn_resnet50_fpn,
)
from torchvision.transforms import functional as F

# 1. Load the pre-trained Faster R-CNN model
weights = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn(weights=weights, box_score_thresh=0.8)
model.eval()  # Set model to evaluation mode
```

Postprocessing Steps

Post-Processing of Detections





Cat: 5%	Dog: 93%	…	Background: 2%
Cat: 87%	Dog: 3%	…	Background: 10%
Cat: 22%	Dog: 15%	…	Background: 63%
⋮			
Cat: 0%	Dog: 4%	…	Background: 96%

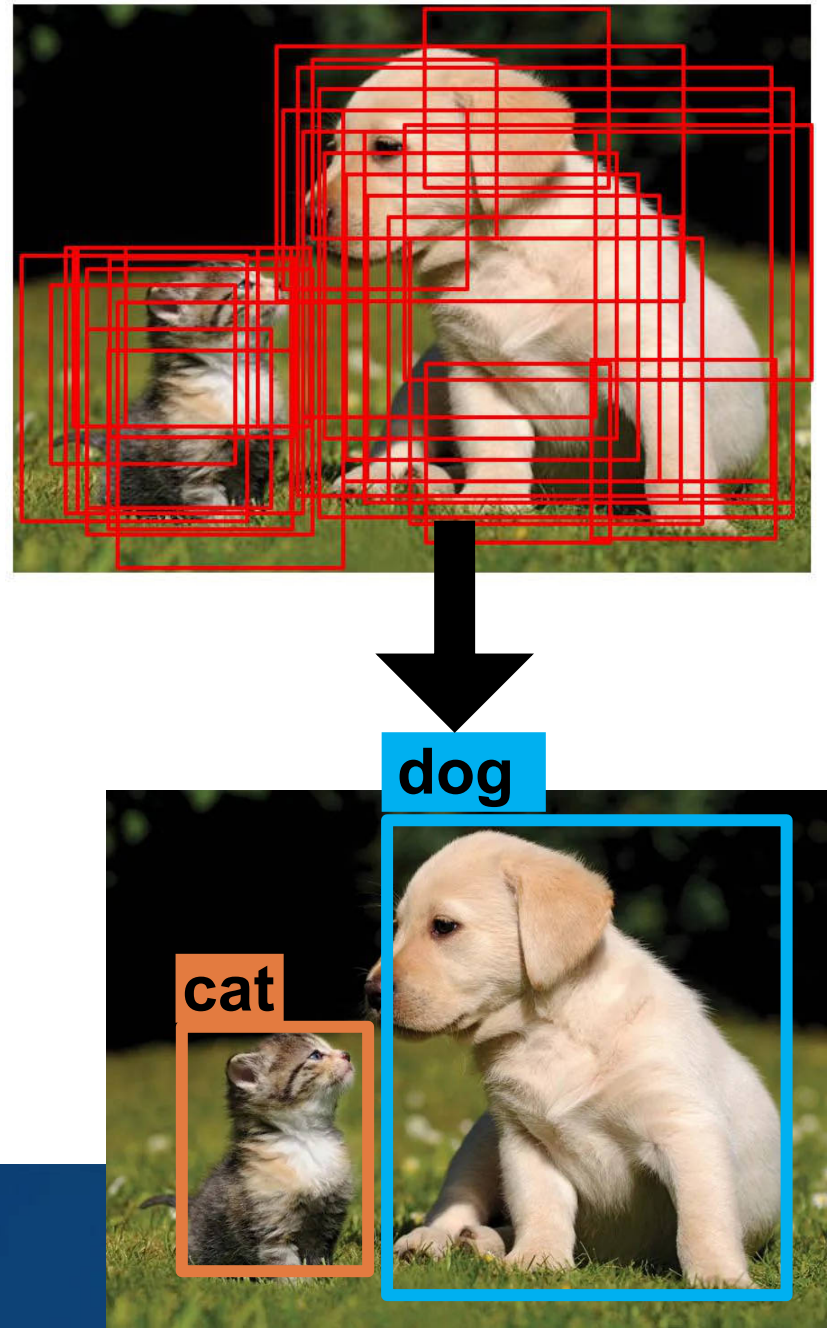
CRICOS No.00213J

Post-Processing Detections

For many object detectors, raw predictions are converted into usable detections by:

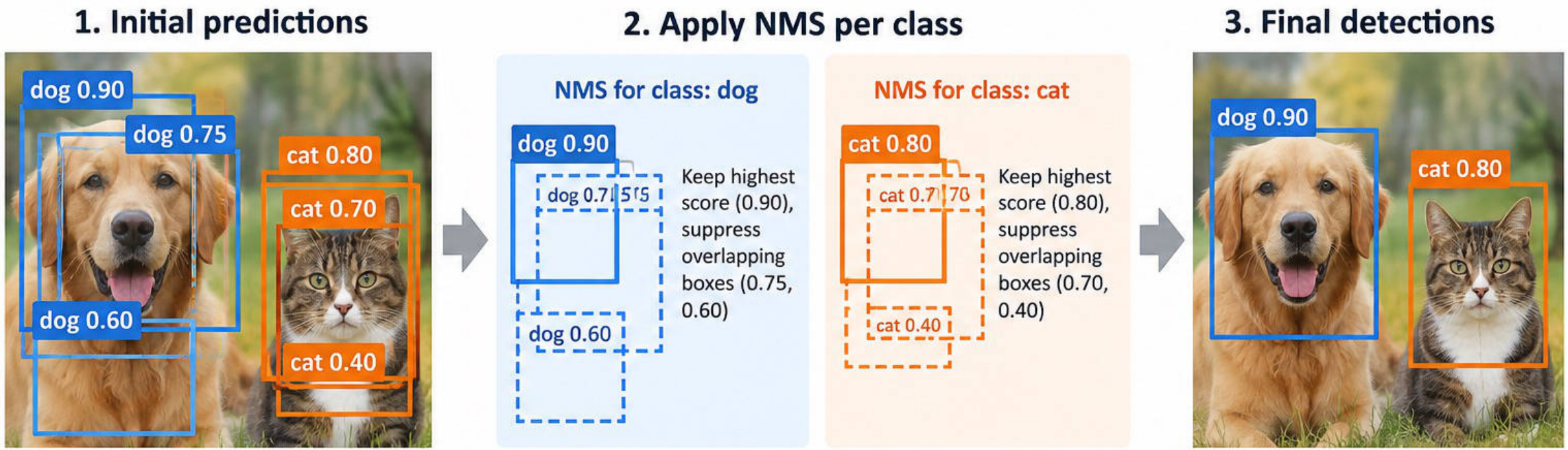
1. **Discard low-confidence predictions**
2. **Remove duplicate detections** using Non-Maximum Suppression (NMS)

DETR-style transformer detectors generally do not require NMS, because one-to-one matching during training encourages different object queries to predict different objects.



Non-Maximum Supression

NMS removes overlapping boxes that predict the same class, keeping the highest-scoring one.
Boxes of different classes are not suppressed, even if they overlap.



CRICOS No.00213J

Illustration by GPT-5.6

Single Stage Detectors

Single-Stage Detectors

Idea: no separate proposal stage, go directly from image to detections

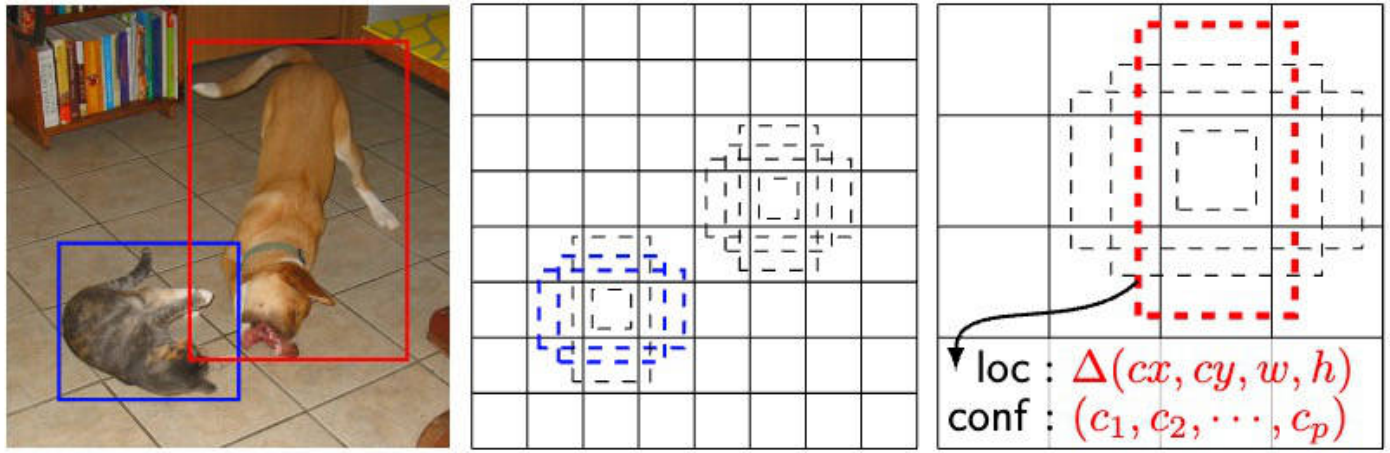
Two Types:

- Anchor-based methods: use pre-defined set of reference boxes and predict refinement + class scores.
  - Historical: SSD, RetinaNet, EfficientDet.
  - Currently less dominant than in the past.

- Point-based methods: every pixel predicts class scores and box geometry.
  - Historical: FCOS, CornerNet, CenterNet.
  - Current: YOLOv8, YOLO11, RTMDet

CRICOS No.00213J

Single-Stage Detectors



(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

Fig. 1: **SSD framework.** (a) SSD only needs an input image and ground truth boxes for each object during training. In a convolutional fashion, we evaluate a small set (e.g. 4) of default boxes of different aspect ratios at each location in several feature maps with different scales (e.g. 8×8 and 4×4 in (b) and (c)). For each default box, we predict both the shape offsets and the confidences for all object categories ($(c_1, c_2, \cdots, c_p)$). At training time, we first match these default boxes to the ground truth boxes. For example, we have matched two default boxes with the cat and one with the dog, which are treated as positives and the rest as negatives. The model loss is a weighted sum between localization loss (e.g. Smooth L1 [6]) and confidence loss (e.g. Softmax).

arXiv:1512.02325v5 [cs.CV] 29 Dec 2016

SSD: Single Shot MultiBox Detector

Wei Liu[1], Dragomir Anguelov[2], Dumitru Erhan[3], Christian Szegedy[3], Scott Reed[4], Cheng-Yang Fu[1], Alexander C. Berg[1]

[1]UNC Chapel Hill [2]Zoox Inc. [3]Google Inc. [4]University of Michigan, Ann-Arbor
[1]wliu@cs.unc.edu, [2]drago@zoox.com, [3]{dumitru,szegedy}@google.com,
[4]reedscot@umich.edu, [1]{cyfu,aberg}@cs.unc.edu

Abstract. We present a method for detecting objects in images using a single deep neural network. Our approach, named SSD, discretizes the output space of bounding boxes into a set of default boxes over different aspect ratios and scales per feature map location. At prediction time, the network generates scores for the presence of each object category in each default box and produces adjustments to the box to better match the object shape. Additionally, the network combines predictions from multiple feature maps with different resolutions to naturally handle objects of various sizes. SSD is simple relative to methods that require object proposals because it completely eliminates proposal generation and subsequent pixel or feature resampling stages and encapsulates all computation in a single network. This makes SSD easy to train and straightforward to integrate into systems that require a detection component. Experimental results on the PASCAL VOC, COCO, and ILSVRC datasets confirm that SSD has competitive accuracy to methods that utilize an additional object proposal step and is much faster, while providing a unified framework for both training and inference. For 300×300 input, SSD achieves 74.3% mAP[1] on VOC2007 `test` at 59 FPS on a Nvidia Titan X and for 512×512 input, SSD achieves 76.9% mAP, outperforming a comparable state-of-the-art Faster R-CNN model. Compared to other single stage methods, SSD has much better accuracy even with a smaller input image size. Code is available at: `https://github.com/weiliu89/caffe/tree/ssd`.

Keywords: Real-time Object Detection; Convolutional Neural Network

1 Introduction

Current state-of-the-art object detection systems are variants of the following approach: hypothesize bounding boxes, resample pixels or features for each box, and apply a high-quality classifier. This pipeline has prevailed on detection benchmarks since the Selective Search work [1] through the current leading results on PASCAL VOC, COCO, and ILSVRC detection all based on Faster R-CNN[2] albeit with deeper features such as [3]. While accurate, these approaches have been too computationally intensive for embedded systems and, even with high-end hardware, too slow for real-time applications.

[1] We achieved even better results using an improved data augmentation scheme in follow-on experiments: 77.2% mAP for 300×300 input and 79.8% mAP for 512×512 input on VOC2007. Please see Sec. 3.6 for details.

Point-based Detection

Almost all state-of-the-art object detectors such as RetinaNet, SSD, YOLOv3, and Faster R-CNN rely on pre-defined anchor boxes. In contrast, our proposed detector FCOS is anchor box free, as well as proposal free. By eliminating the predefined set of anchor boxes, FCOS completely avoids the complicated computation related to anchor boxes such as calculating overlapping during training. More importantly, we also avoid all hyper-parameters related to anchor boxes, which are often very sensitive to the final detection performance. […]
For the first time, we demonstrate a much simpler and flexible detection framework achieving improved detection accuracy.

arXiv:1904.01355v5 [cs.CV] 20 Aug 2019

FCOS: Fully Convolutional One-Stage Object Detection

Zhi Tian Chunhua Shen* Hao Chen Tong He
The University of Adelaide, Australia

Abstract

We propose a fully convolutional one-stage object detector (FCOS) to solve object detection in a per-pixel prediction fashion, analogue to semantic segmentation. Almost all state-of-the-art object detectors such as RetinaNet, SSD, YOLOv3, and Faster R-CNN rely on pre-defined anchor boxes. In contrast, our proposed detector FCOS is anchor box free, as well as proposal free. By eliminating the pre-defined set of anchor boxes, FCOS completely avoids the complicated computation related to anchor boxes such as calculating overlapping during training. More importantly, we also avoid all hyper-parameters related to anchor boxes, which are often very sensitive to the final detection performance. With the only post-processing non-maximum suppression (NMS), FCOS with ResNeXt-64x4d-101 achieves 44.7% in AP with single-model and single-scale testing, surpassing previous one-stage detectors with the advantage of being much simpler. For the first time, we demonstrate a much simpler and flexible detection framework achieving improved detection accuracy. We hope that the proposed FCOS framework can serve as a simple and strong alternative for many other instance-level tasks. Code is available at:
tinyurl.com/FCOSv1

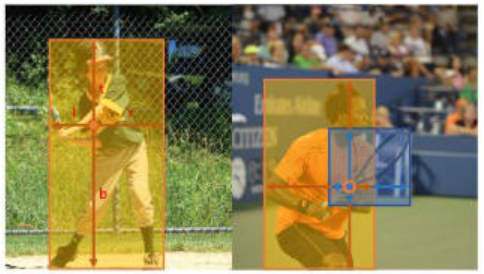


Figure 1 – As shown in the left image, FCOS works by predicting a 4D vector (l, t, r, b) encoding the location of a bounding box at each foreground pixel (supervised by ground-truth bounding box information during training). The right plot shows that when a location residing in multiple bounding boxes, it can be ambiguous in terms of which bounding box this location should regress.

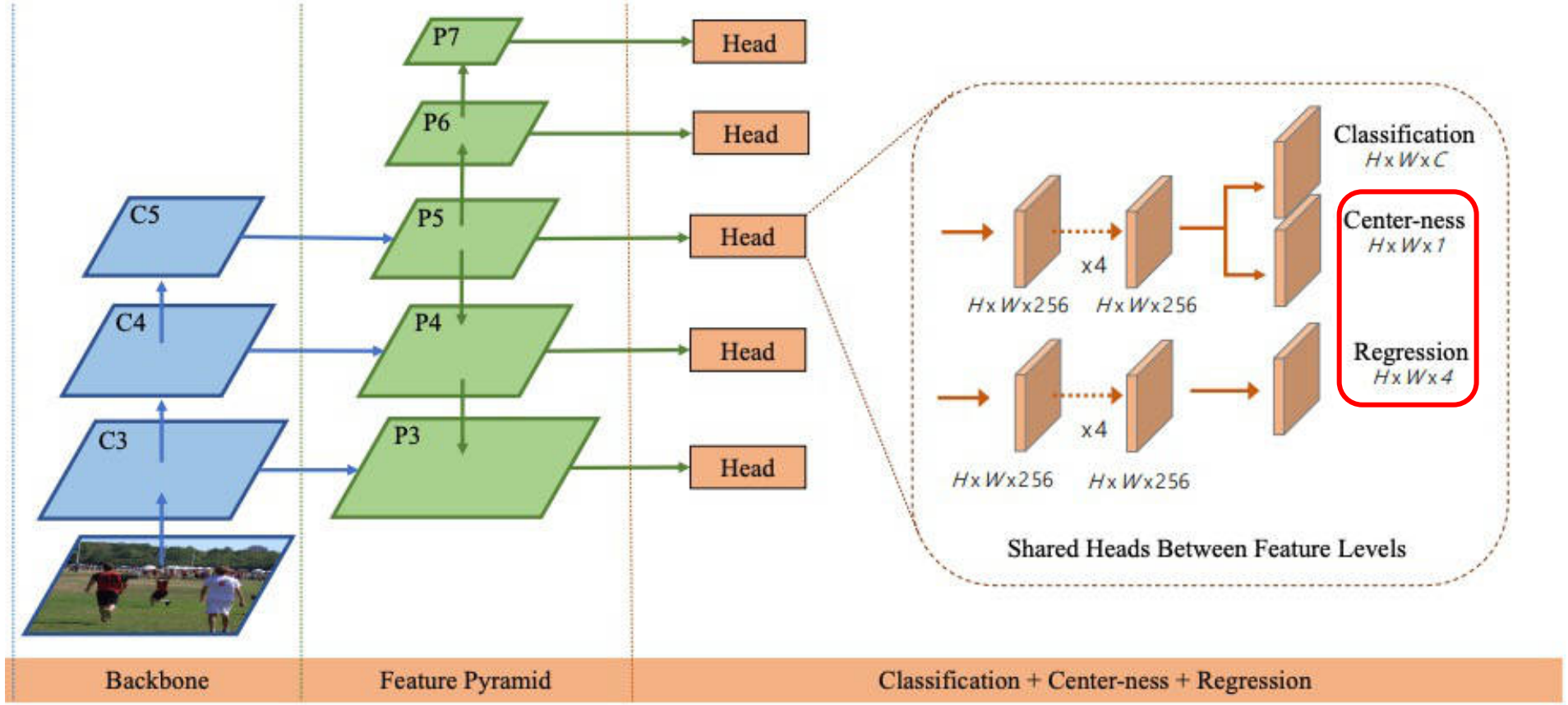
1. Introduction

Object detection is a fundamental yet challenging task in computer vision, which requires the algorithm to predict a bounding box with a category label for each instance of interest in an image. All current mainstream detectors such as Faster R-CNN [24], SSD [18] and YOLOv2, v3 [23] rely on a set of pre-defined anchor boxes and *it has long been believed that the use of anchor boxes is the key to detectors' success*. Despite their great success, it is important to note that anchor-based detectors suffer some drawbacks: 1) As shown in [15, 24], detection performance is sensitive to the sizes, aspect ratios and number of anchor boxes. For example, in RetinaNet [15], varying these hyper-parameters affects the performance up to 4% in AP on the COCO benchmark [16]. As a result, these hyper-parameters need to be carefully tuned in anchor-based detectors. 2) Even with careful design, because the scales and aspect ratios of anchor boxes are kept fixed, detectors encounter difficulties to deal with object candidates with large shape variations, particularly for small objects. The pre-defined anchor boxes also hamper the generalization ability of detectors, as they need to be re-designed on new detection tasks with different object sizes or aspect ratios. 3) In order to achieve a high recall rate, an anchor-based detector is required to densely place anchor boxes on the input image (*e.g.*, more than 180K anchor boxes in feature pyramid networks (FPN) [14] for an image with its shorter side being 800). Most of these anchor boxes are labelled as negative samples during training. The excessive number of negative samples aggravates the imbalance between positive and negative samples in training. 4) Anchor boxes also involve complicated computation such as calculating the intersection-over-union (IoU) scores with ground-truth bounding boxes.

Recently, fully convolutional networks (FCNs) [20] have achieved tremendous success in dense prediction tasks such as semantic segmentation [20, 28, 9, 19], depth estimation

*Corresponding author, email: chunhua.shen@adelaide.edu.au

Point-based Detection (FCOS)



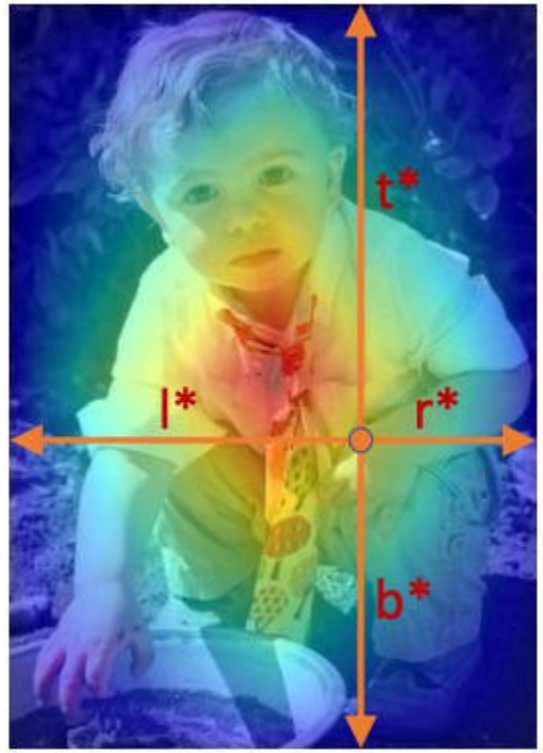
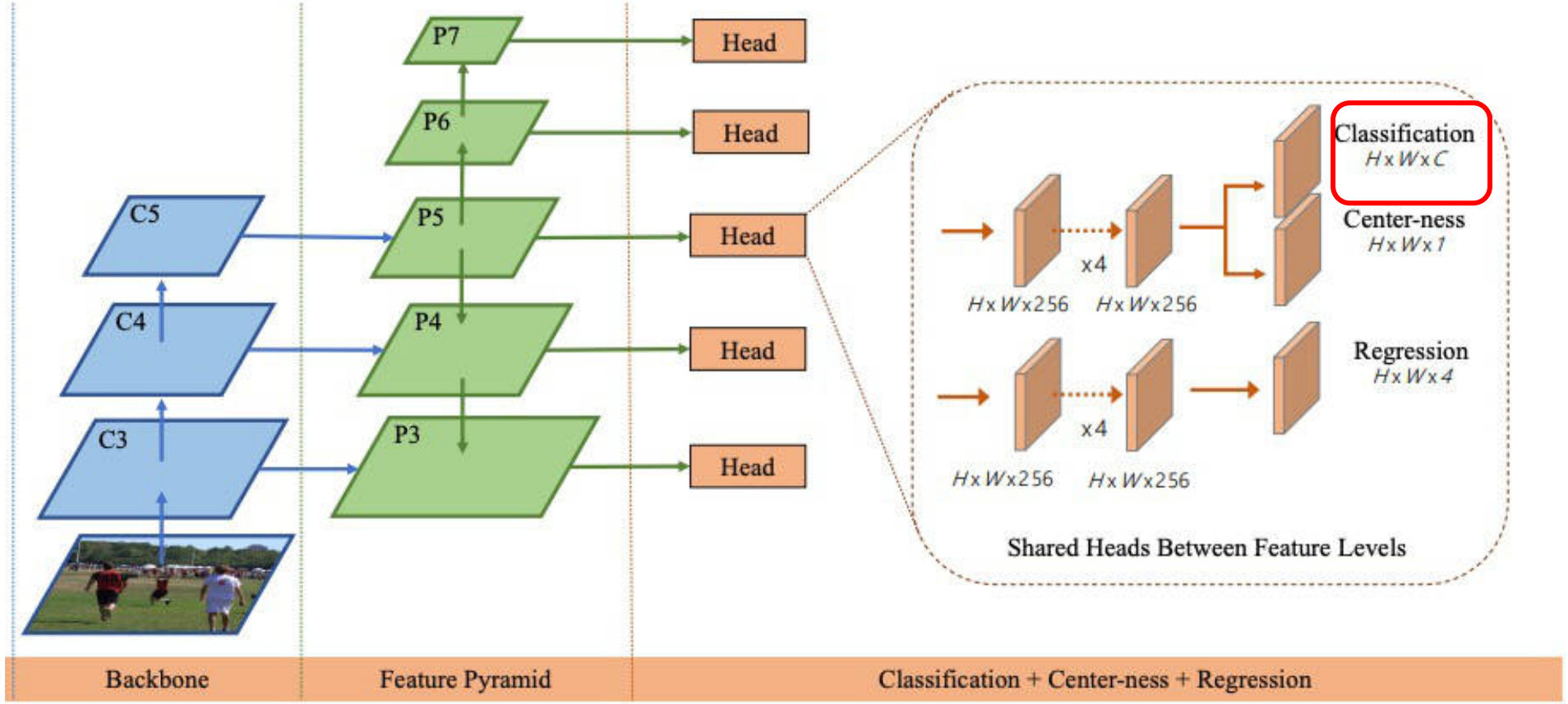
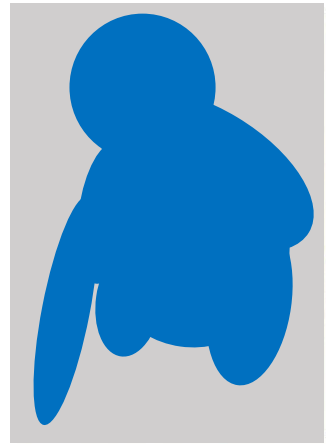


Illustration: FCOS

Illustrations / slide concept by Dr Dimity Miller

Point-based Detection (FCOS)



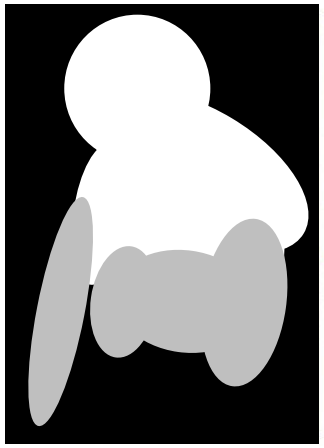


Person

Background

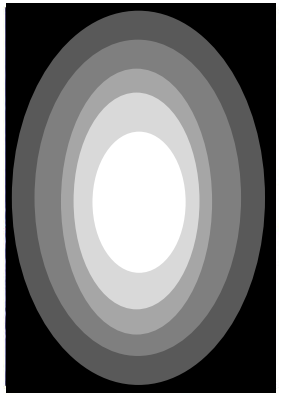
Point-based Detection (FCOS)

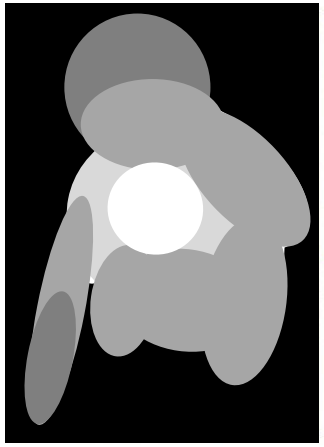
Classification scores for
a class, e.g. "person"



Multiply both masks

Centerness scores



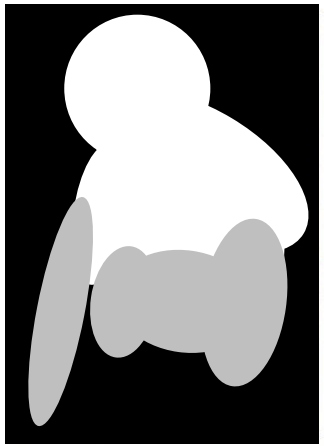


Centerness-aware
classification scores for
class "person"

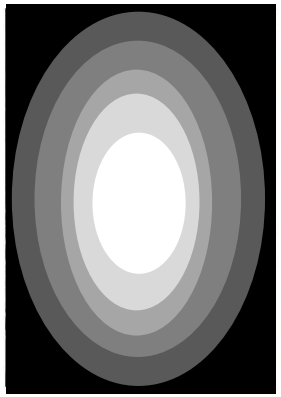
Illustrations / slide concept by Dr Dimity Miller

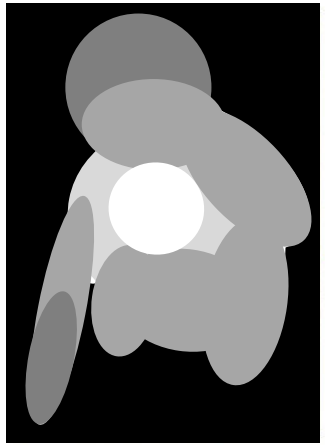
Point-based Detection (FCOS)

Classification scores for a class, e.g. "person"



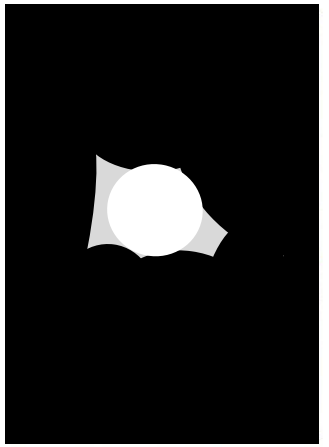
Centerness scores





Centerness-aware classification scores for class "person"

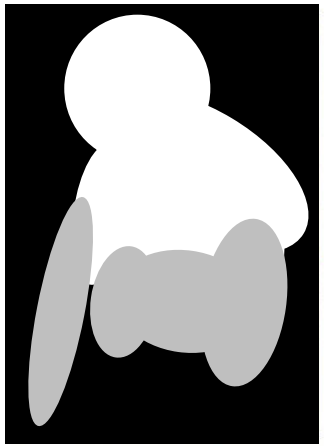
Remove low confidence pixels



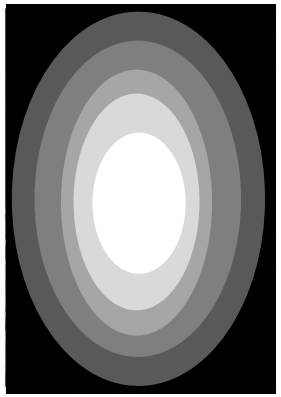
Illustrations / slide concept by Dr Dimity Miller

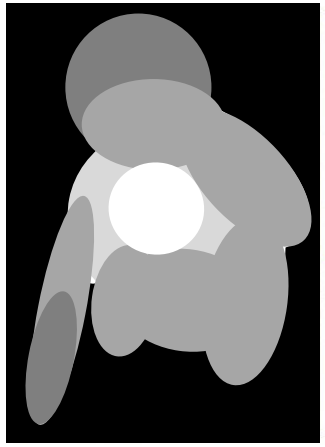
Point-based Detection (FCOS)

Classification scores for a class, e.g. "person"



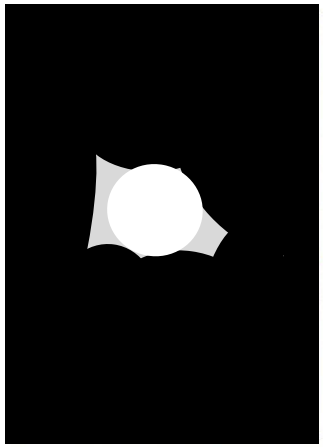
Centerness scores





Centerness-aware classification scores for class "person"

Remove low confidence pixels



Regression prediction for every pixel

Bounding boxes with associated scores for class "person"

Illustrations / slide concept by Dr Dimity Miller

Transformer-based Detectors

DETR – End-to-End Object Detection

Key idea: Instead of generating many candidate boxes and cleaning them up afterwards, predict a **fixed-size set of object hypotheses directly**.

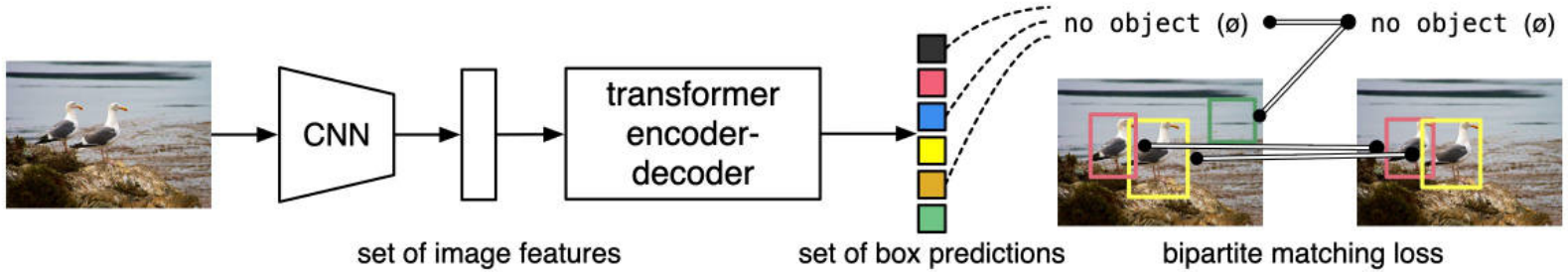


Fig. 1: DETR directly predicts (in parallel) the final set of detections by combining a common CNN with a transformer architecture. During training, bipartite matching uniquely assigns predictions with ground truth boxes. Prediction with no match should yield a "`no object`" (∅) class prediction.

arXiv:2005.12872v3 [cs.CV] 28 May 2020

End-to-End Object Detection with Transformers

Nicolas Carion*, Francisco Massa*, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko

Facebook AI

Abstract. We present a new method that views object detection as a direct set prediction problem. Our approach streamlines the detection pipeline, effectively removing the need for many hand-designed components like a non-maximum suppression procedure or anchor generation that explicitly encode our prior knowledge about the task. The main ingredients of the new framework, called DEtection TRansformer or DETR, are a set-based global loss that forces unique predictions via bipartite matching, and a transformer encoder-decoder architecture. Given a fixed small set of learned object queries, DETR reasons about the relations of the objects and the global image context to directly output the final set of predictions in parallel. The new model is conceptually simple and does not require a specialized library, unlike many other modern detectors. DETR demonstrates accuracy and run-time performance on par with the well-established and highly-optimized Faster R-CNN baseline on the challenging COCO object detection dataset. Moreover, DETR can be easily generalized to produce panoptic segmentation in a unified manner. We show that it significantly outperforms competitive baselines. Training code and pretrained models are available at https://github.com/facebookresearch/detr.

1 Introduction

The goal of object detection is to predict a set of bounding boxes and category labels for each object of interest. Modern detectors address this set prediction task in an indirect way, by defining surrogate regression and classification problems on a large set of proposals [37,5], anchors [23], or window centers [53,46]. Their performances are significantly influenced by postprocessing steps to collapse near-duplicate predictions, by the design of the anchor sets and by the heuristics that assign target boxes to anchors [52]. To simplify these pipelines, we propose a direct set prediction approach to bypass the surrogate tasks. This end-to-end philosophy has led to significant advances in complex structured prediction tasks such as machine translation or speech recognition, but not yet in object detection: previous attempts [43,16,4,39] either add other forms of prior knowledge, or have not proven to be competitive with strong baselines on challenging benchmarks. This paper aims to bridge this gap.

* Equal contribution

DETR

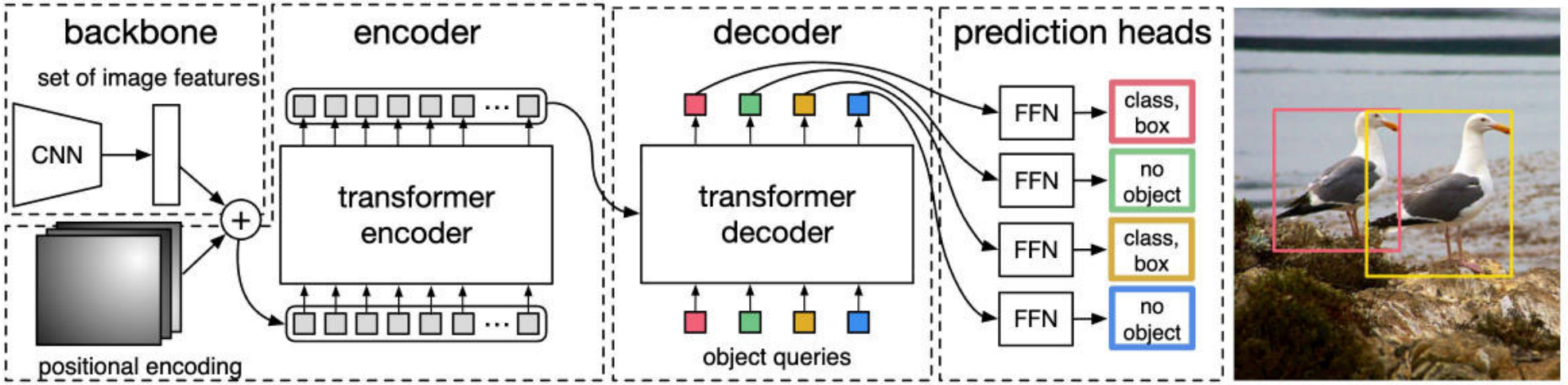


Fig. 2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a "`no object`" class.

Why this was a big deal

DETR made the object detection pipeline much more end-to-end.

One-Stage detectors	DETR-style
Anchors / points	Object queries
Thousands of candidates	Small fixed set of queries
Many duplicate predictions	One-to-one predictions
Requires NMS	Typically no NMS
Local dense prediction	Global attention / set prediction

CRICOS No.00213J

DETR – Modern Variants

Original DETR (2020) suffered from:

- **slow training convergence**;
- difficulty with **small objects**;
- high computational cost from global attention over image features.

- **Deformable DETR (2020)**
  - Sparse/deformable attention
  - Better multi-scale and small-object handling
  - Much faster convergence
- **DINO (2022)**
  - Improved query initialization and denoising training
  - Strong modern DETR baseline
  - Uses anchor-like reference boxes internally, but still follows the DETR set-prediction paradigm
- **RT-DETR (2023)**
  - Designed specifically for **real-time end-to-end detection**
  - Keeps the no-NMS DETR formulation while targeting practical inference speed
  - Latest: **RT-DETRv3 (WACV 2025)**

- **Recent trend:** better matching, denser training supervision, and real-time efficiency.
- If interested, check DEIM (CVPR 2025), MI-DETR (CVPR 2025), RT-DETRv4 (2025), Association DETR and SA-Matching DETR (2026)

CRICOS No.00213J

Training of Object Detectors

Training

- Object Detection = Classification + Localisation + Assignment
- For each prediction, the model has to learn:
  - **what** the object is;
  - **where** the object is;
  - **which** prediction should be responsible for which ground-truth object.

- Losses: Classification Loss + Bounding-Box Regression Loss

CRICOS No.00213J

Training Losses

Classification loss

- For assigned object predictions, train the model to predict the correct class.
- Typical losses:
  - cross-entropy / softmax;
  - binary cross-entropy / sigmoid;
  - focal loss for heavily imbalanced dense detectors.

- Classification Loss penalises incorrect object labels and false background predictions.

CRICOS No.00213J

Training Losses

Bounding-box Regression Loss

- Train predicted boxes to match the ground-truth geometry.
- Simple coordinate losses such as L1, but modern detectors often use IoU-based losses such as: 1 – IoU or variants such as GIoU, DIoU, CIoU.

- Coordinate losses ask whether the box parameters are close.
  IoU losses ask whether the **boxes themselves overlap well**.

Training Losses

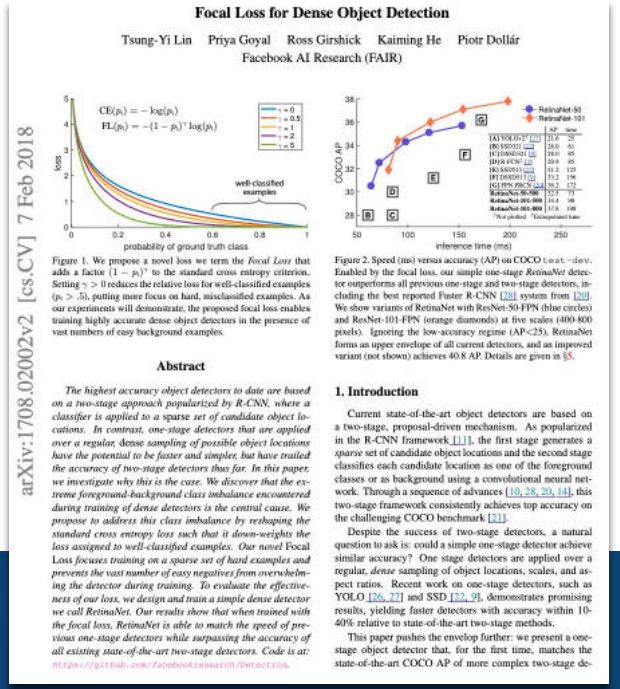
Objectness / foreground-background loss

- Some detectors separately ask: “Is there an object here at all?”
- For example:
  - Faster R-CNN's RPN predicts **objectness**;
  - YOLO-style architectures have often used objectness-style predictions.
- This can be separate from the class prediction.

CRICOS No.00213J

Major Challenge: Class Imbalance

- Images contain typically only few objects but potentially thousands of candidate predictions, most of which are of the *background* class.
- Easy background examples can dominate the training loss.
- Different detectors deal with this differently, but common solutions include
  - Careful sampling strategy
  - Focal loss (down-weights easy, correctly classified examples so that training focuses on hard mistakes)
  - one-to-one set matching in DETR



Focal Loss for Dense Object Detection

Tsung-Yi Lin Priya Goyal Ross Girshick Kaiming He Piotr Dollár

Facebook AI Research (FAIR)

arXiv:1708.02002v2 [cs.CV] 7 Feb 2018

Figure 1. We propose a novel loss we term the *Focal Loss* that adds a factor $(1 - p_t)^\gamma$ to the standard cross entropy criterion. Setting $\gamma > 0$ reduces the relative loss for well-classified examples ($p_t > .5$), putting more focus on hard, misclassified examples. As our experiments will demonstrate, the proposed focal loss enables training highly accurate dense object detectors in the presence of vast numbers of easy background examples.

Figure 2. Speed (ms) versus accuracy (AP) on COCO test-dev. Enabled by the focal loss, our simple one-stage *RetinaNet* detector outperforms all previous one-stage and two-stage detectors, including the best reported Faster R-CNN [28] system from [20]. We show variants of RetinaNet with ResNet-50-FPN (blue circles) and ResNet-101-FPN (orange diamonds) at five scales (400-800 pixels). Ignoring the low-accuracy regime (AP<25), RetinaNet forms an upper envelope of all current detectors, and an improved variant (not shown) achieves 40.8 AP. Details are given in §5.

Abstract

The highest accuracy object detectors to date are based on a two-stage approach popularized by R-CNN, where a classifier is applied to a sparse set of candidate object locations. In contrast, one-stage detectors that are applied over a regular, dense sampling of possible object locations have the potential to be faster and simpler, but have trailed the accuracy of two-stage detectors thus far. In this paper, we investigate why this is the case. We discover that the extreme foreground-background class imbalance encountered during training of dense detectors is the central cause. We propose to address this class imbalance by reshaping the standard cross entropy loss such that it down-weights the loss assigned to well-classified examples. Our novel Focal Loss *focuses training on a sparse set of hard examples and prevents the vast number of easy negatives from overwhelming the detector during training. To evaluate the effectiveness of our loss, we design and train a simple dense detector we call RetinaNet. Our results show that when trained with the focal loss, RetinaNet is able to match the speed of previous one-stage detectors while surpassing the accuracy of all existing state-of-the-art two-stage detectors. Code is at:* https://github.com/facebookresearch/detectron.

1. Introduction

Current state-of-the-art object detectors are based on a two-stage, proposal-driven mechanism. As popularized in the R-CNN framework [11], the first stage generates a *sparse* set of candidate object locations and the second stage classifies each candidate location as one of the foreground classes or as background using a convolutional neural network. Through a sequence of advances [10, 28, 20, 14], this two-stage framework consistently achieves top accuracy on the challenging COCO benchmark [21].

Despite the success of two-stage detectors, a natural question to ask is: could a simple one-stage detector achieve similar accuracy? One stage detectors are applied over a regular, *dense* sampling of object locations, scales, and aspect ratios. Recent work on one-stage detectors, such as YOLO [26, 27] and SSD [22, 9], demonstrates promising results, yielding faster detectors with accuracy within 10-40% relative to state-of-the-art two-stage methods.

This paper pushes the envelop further: we present a one-stage object detector that, for the first time, matches the state-of-the-art COCO AP of more complex two-stage de-

Commonly Used Datasets

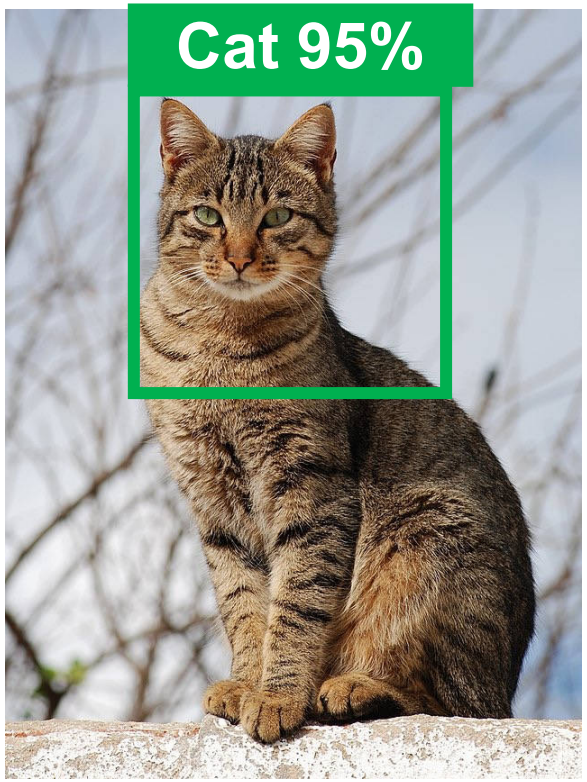
- **COCO:** standard general-purpose benchmark dataset. 80 Classes
- **Objects365:** much larger, 365 classes, 2M images, 30M boxes.
- **OpenImages V7**: very large and broad, 600 classes, 15.9M boxes
- **LVIS:** specifically designed for long-tail distribution: 1200+ classes, many of them rare
- **nuImages:** autonomous driving, large-scale detection and segmentation

CRICOS No.00213J

Evaluating Object Detection Performance

Evaluating Performance



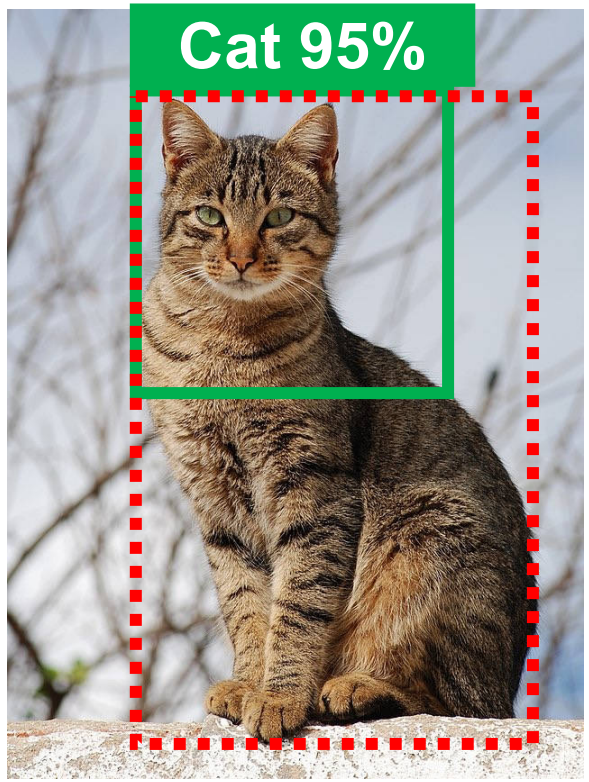


CRICOS No.00213J

Evaluating Performance



IoU = 0.9



IoU = 0.3

- Calulate IoU with ground truth

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

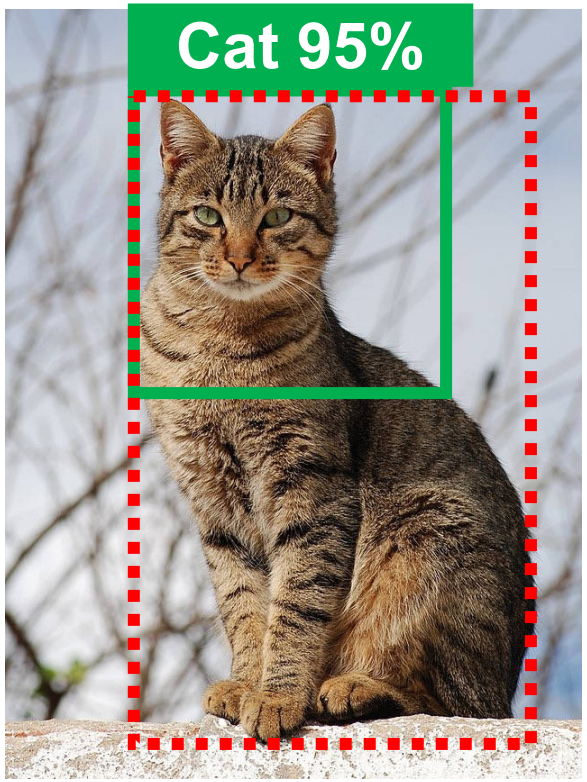


CRICOS No.00213J

Evaluating Performance



IoU = 0.9



IoU = 0.3

- Calulate IoU with ground truth
- If IoU > threshold
  AND correct class wins
  then: **True Positive**
  else: **False Positive**

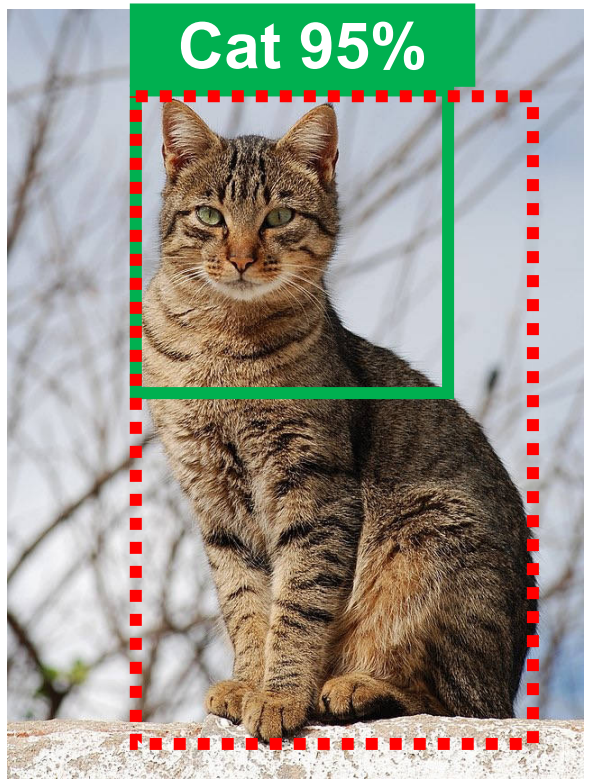
CRICOS No.00213J

Evaluating Performance



IoU = 0.9

→ True positive



IoU = 0.3

→ False positive

… for IoU threshold of 0.5

- Calulate IoU with ground truth
- If IoU > threshold
  AND correct class wins
  then: **True Positive**
  else: **False Positive**

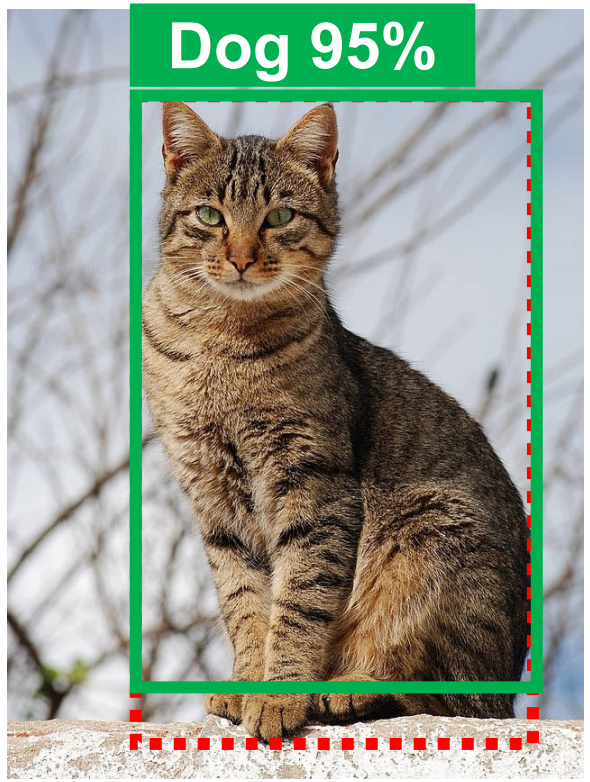
CRICOS No.00213J

Evaluating Performance



IoU = 0.9

→ True positive



IoU = 0.9

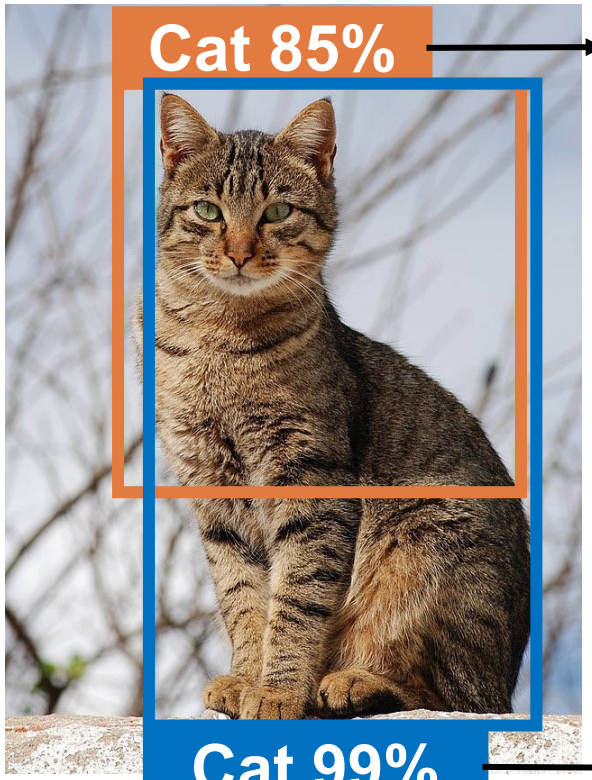
→ False positive

… for IoU threshold of 0.5

- Calulate IoU with ground truth
- If IoU > threshold
  AND correct class wins
  then: **True Positive**
  else: **False Positive**

CRICOS No.00213J

Evaluating Performance



- Calulate IoU with ground truth
- If IoU > threshold
  AND correct class wins
  AND object not already detected
  then: **True Positive**
  else: **False Positive**

CRICOS No.00213J

Evaluating Performance

A detection is a *true positive,* – a true prediction of an object – if:

1. It has an IoU with a GT object greater than (or equal to) the IoU threshold. GT objects previously detected are ignored.
2. It has correctly predicted the object's class.

All detections remaining are *false positives* – a false prediction of an object.

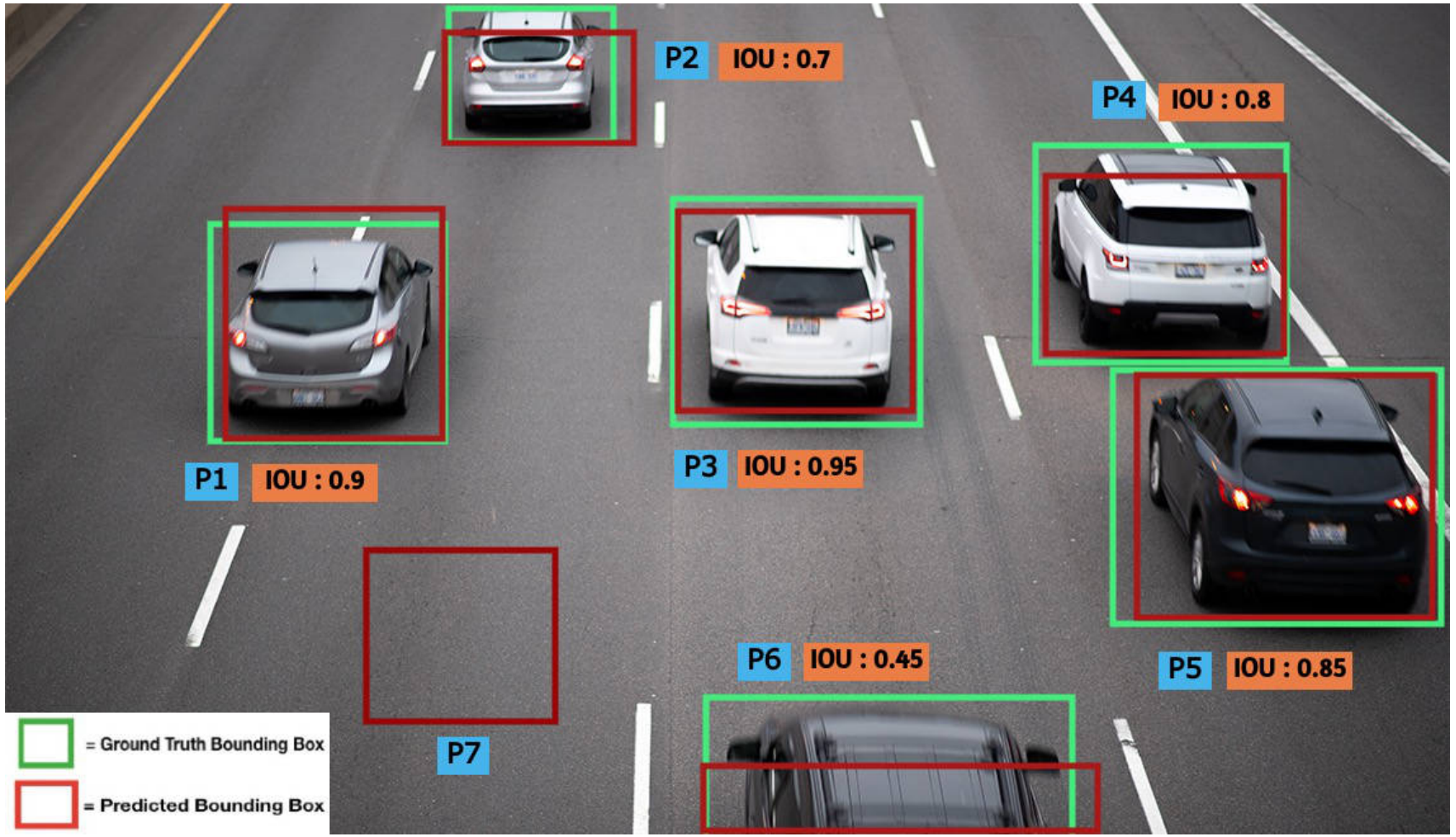
Any GT object that is not detected (i.e. no corresponding true positive detection), is a *false negative* – a false prediction of no object.

CRICOS No.00213J

Evaluating Performance

Which detections are
- True Positives,
- False Positives,
- False Negatives

if the IoU threshold is 0.5?



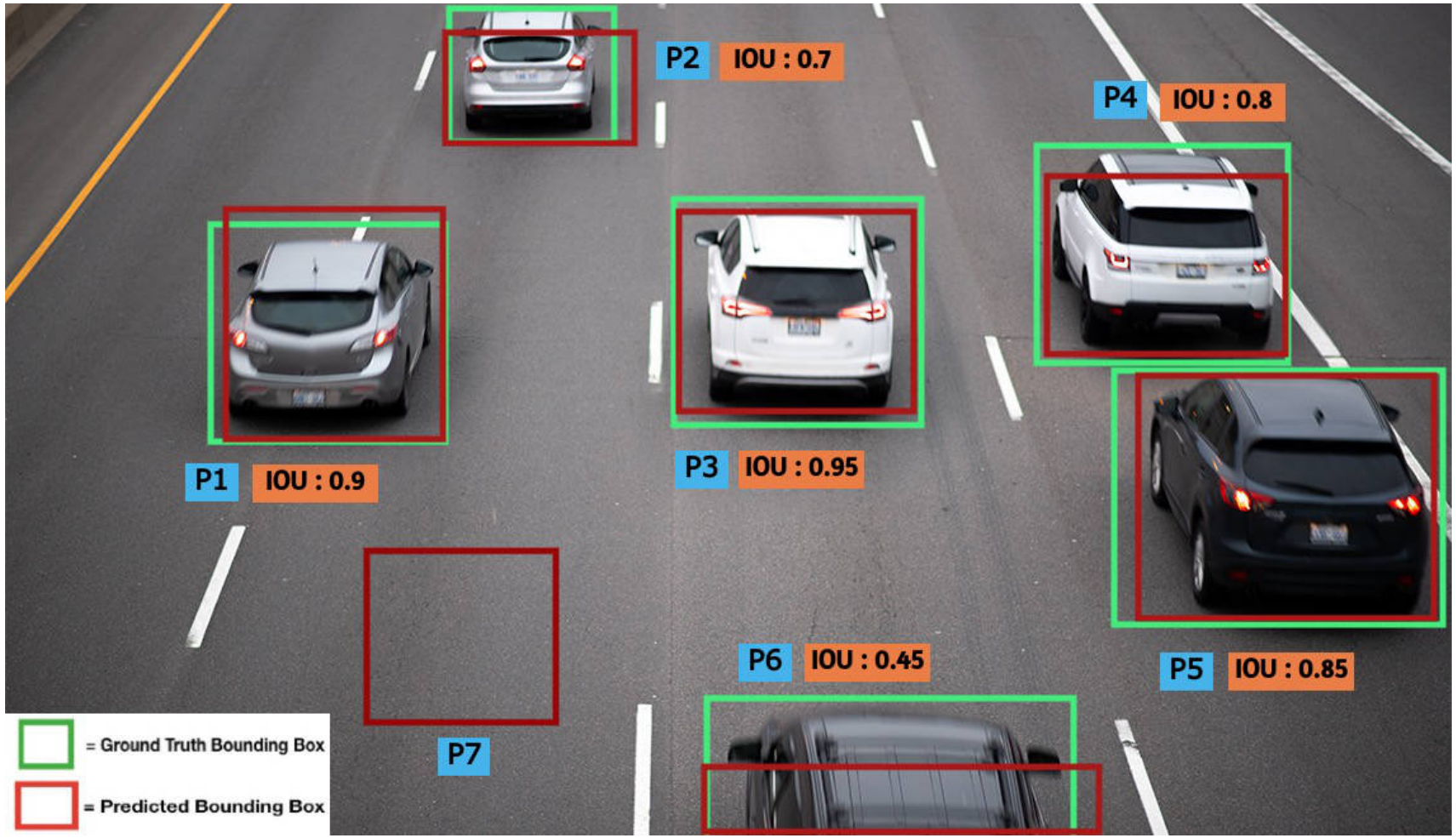
CRICOS No.00213J

Illustrations / slide concept by Dr Dimity Miller

Evaluating Performance

Which detections are
- True Positives,
- False Positives,
- False Negatives

if the IoU threshold is 0.75?



CRICOS No.00213J

Evaluating Performance

From TP, FP, FN we can calculate precision and recall per class

- **Precision** = $\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$
  - *"What fraction of the detections is correct?"*
- **Recall** = $\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}} = \frac{\mathrm{TP}}{\mathrm{P}}$
  - *"What fraction of the actual objects were correctly detected?"*

CRICOS No.00213J

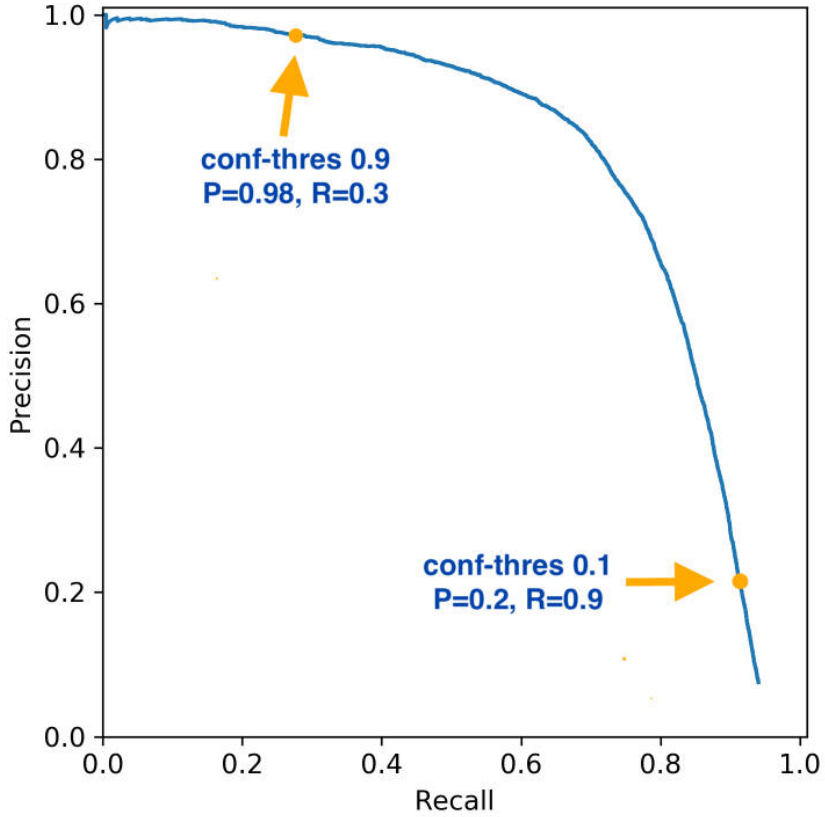
Evaluating Performance

From TP, FP, FN we can calculate precision and recall per class

- **Precision** = $\frac{\mathrm{TP}}{\mathrm{TP+FP}}$
  - *"What fraction of the detections is correct?"*
- **Recall** = $\frac{\mathrm{TP}}{\mathrm{TP+FN}} = \frac{\mathrm{TP}}{\mathrm{P}}$
  - *"What fraction of the actual objects were correctly detected?"*

TP, FP, FN depend on the thresholds:
(1) the confidence threshold and (2) the IoU threshold.

We can draw a Precision-Recall curve by varying the confidence threshold



CRICOS No.00213J

Evaluating Performance

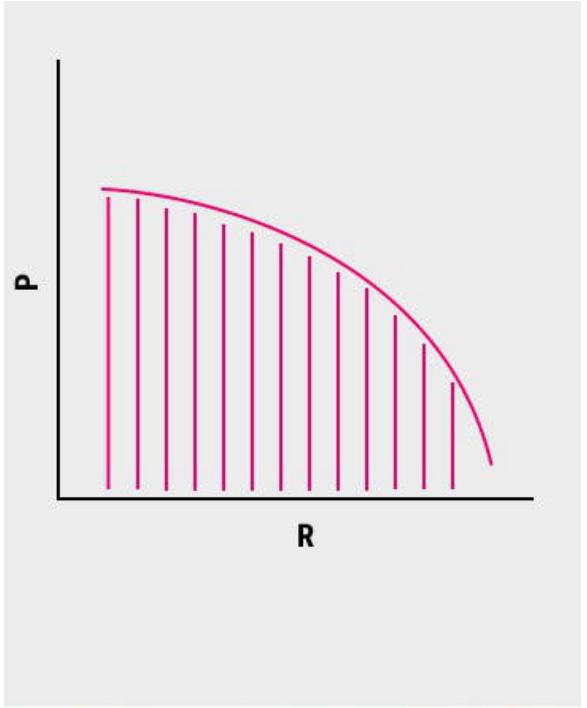
Create a summary measure of performance for one particular class at one particular IoU threshold:

Average Precision (AP): is the area under the PR curve.

- Sample the PR curve at regular values of recall (e.g. 101), average the corresponding precision values

mean Average Precision (mAP): is the mean AP over all k classes in the dataset.

$$\mathrm{mAP} = \frac{1}{k}\sum_{\mathrm{i}=1}^{\mathrm{k}} \mathrm{AP_i}$$



CRICOS No.00213J

Evaluating Performance

Summary performance measure over the whole dataset: average mAP for different IoU thresholds

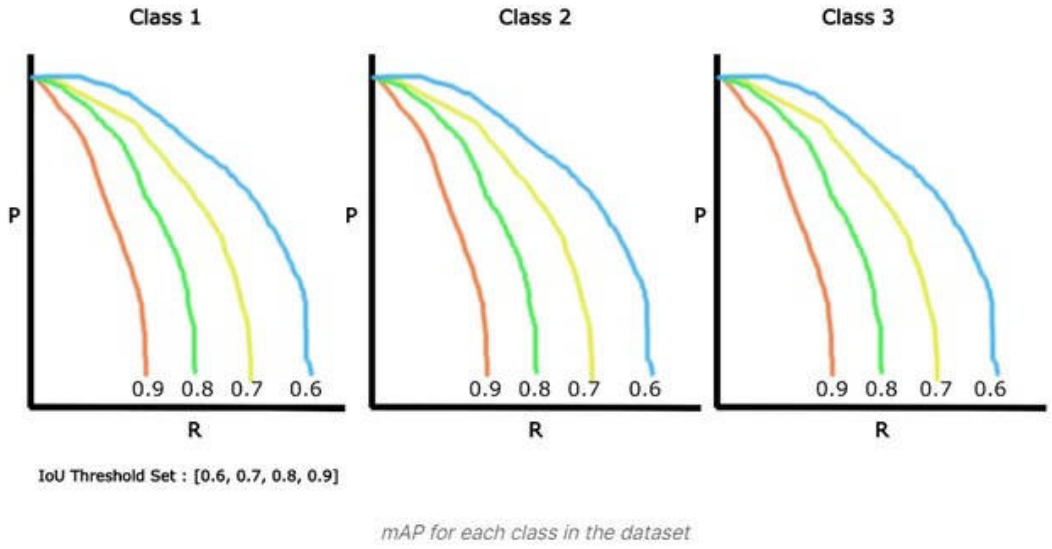
This is the standard evaluation metric, introduced in COCO. It is (confusingly) also called mAP.

Best performance occurs when:

- All objects are detected with very high IoU and classified correctly with high confidence
- There are no false positives, or they are very low confidence

mAP can mean two things:

- mean AP across all classes at a fixed IoU threshold, e.g. mAP@0.5
- mean AP across all classes averaged over different IoU thresholds , e.g. mAP@[0.5:0.95]

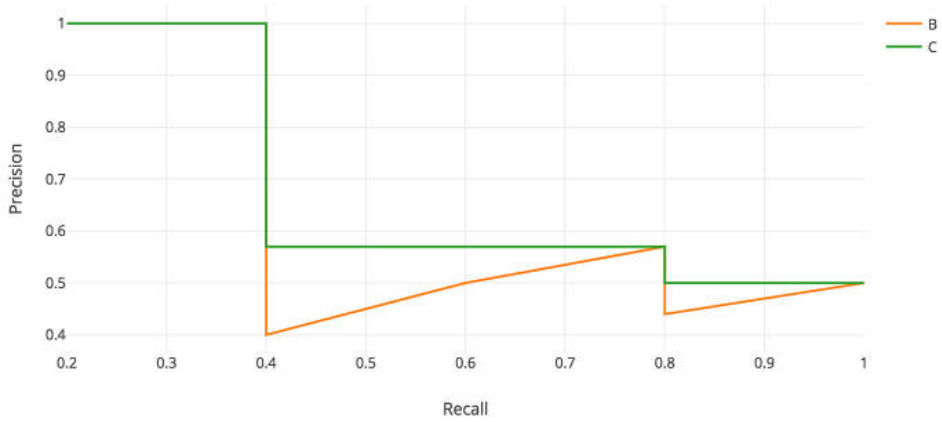


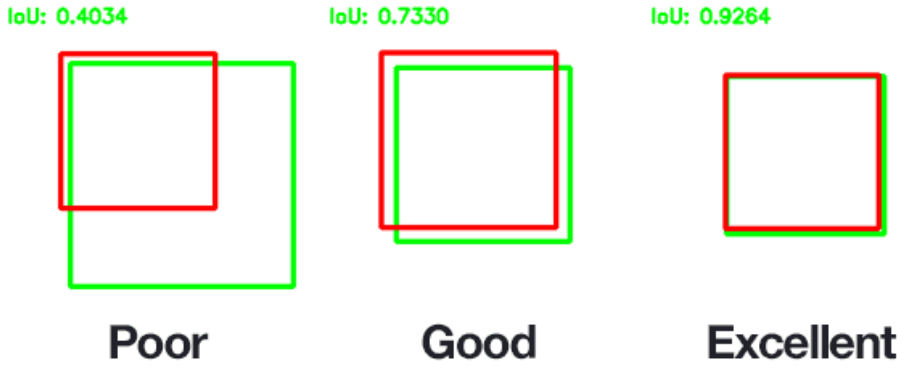
mAP for each class in the dataset

Evaluating Performance

Things to be aware of when interpreting mAP for robotics:

- **AP depends on how the precision-recall curve is processed**: AP summarises performance over many confidence thresholds, usually using interpolation/smoothing of the PR curve. Two detectors with similar behaviour at the threshold you actually deploy may still have different AP.
- **mAP penalises low recall strongly:** Missing objects reduces the achievable area under the PR curve across a wide range of thresholds. In many robotics tasks, a detector that misses safety-critical objects can therefore be heavily penalised – often appropriately – but this may not reflect the exact deployment trade-off between false positives and false negatives.
- **IoU may be stricter than the application requires:** A prediction can be counted as wrong because its bounding box does not overlap the ground truth enough, even if it is still good enough for the robot's downstream task.
- **mAP gives every class equal weight:** Dataset class frequencies can be highly imbalanced, but mAP averages AP equally across classes. A rare class contributes just as much as a very common class. This is useful for avoiding domination by frequent classes, but it may not match the real-world importance or frequency of different objects.





CRICOS No.00213J

Evaluating Performance

- mAP is a useful benchmark metric, but not necessarily the metric that best reflects robotic task performance**.**
- For robotics, you may also care about things like **recall at a fixed false-positive rate**, **latency**, **distance-dependent recall**, or performance on particular safety-critical classes.
- It's important to look into performance more deeply – e.g. depending on your application, what types of errors are really dangerous, and which are tolerable?

CRICOS No.00213J

Open Set and Open Vocabulary Detection

Closed Set Training

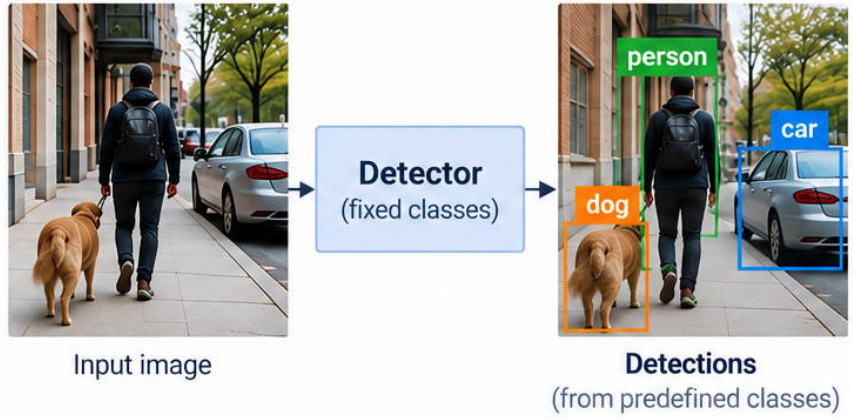
How are object detectors trained?

CRICOS No.00213J

Closed Set Training

How are object detectors trained?

- Dataset of annotated bounding boxes and class labels
- There is a **fixed list** of object classes that are annotated (e.g. person, bicycle, car, dog, cat, backpack, umbrella, chair, …)
  - Depends on the dataset used, e.g. 80 different classes in COCO



Example fixed class sets:

e.g. COCO (80 classes):

```
person, bicycle, car, dog, cat, ….
backpack, umbrella, chair, ….
```

- Can only detect objects within the predefined set
- Need to retrain to add new classes

CRICOS No.00213J

Closed Set Training vs Deployment

How are object detectors trained?

- Dataset of annotated bounding boxes and class labels
- There is a **fixed list** of object classes that are annotated (e.g. person, bicycle, car, dog, cat, backpack, umbrella, chair, …)
  - Depends on the dataset used, e.g. 80 different classes in COCO

How are object detectors deployed?

CRICOS No.00213J

Closed Set Training vs Deployment

How are object detectors trained?

- Dataset of annotated bounding boxes and class labels
- There is a **fixed list** of object classes that are annotated (e.g. person, bicycle, car, dog, cat, backpack, umbrella, chair, …)
  - Depends on the dataset used, e.g. 80 different classes in COCO

How are object detectors deployed?

- They might encounter objects they were never trained on!
- Common failure mode: detect one of those new objects and classify it as one of the known classes

CRICOS No.00213J

Closed Set & Open Set Conditions

Closed Set

All classes that may appear at test time are known during training.

Detector: Choose among fixed known classes

Open Set

Unknown classes may appear at test time.

Detector: Detect known classes and identify novel objects as **unknown**

CRICOS No.00213J

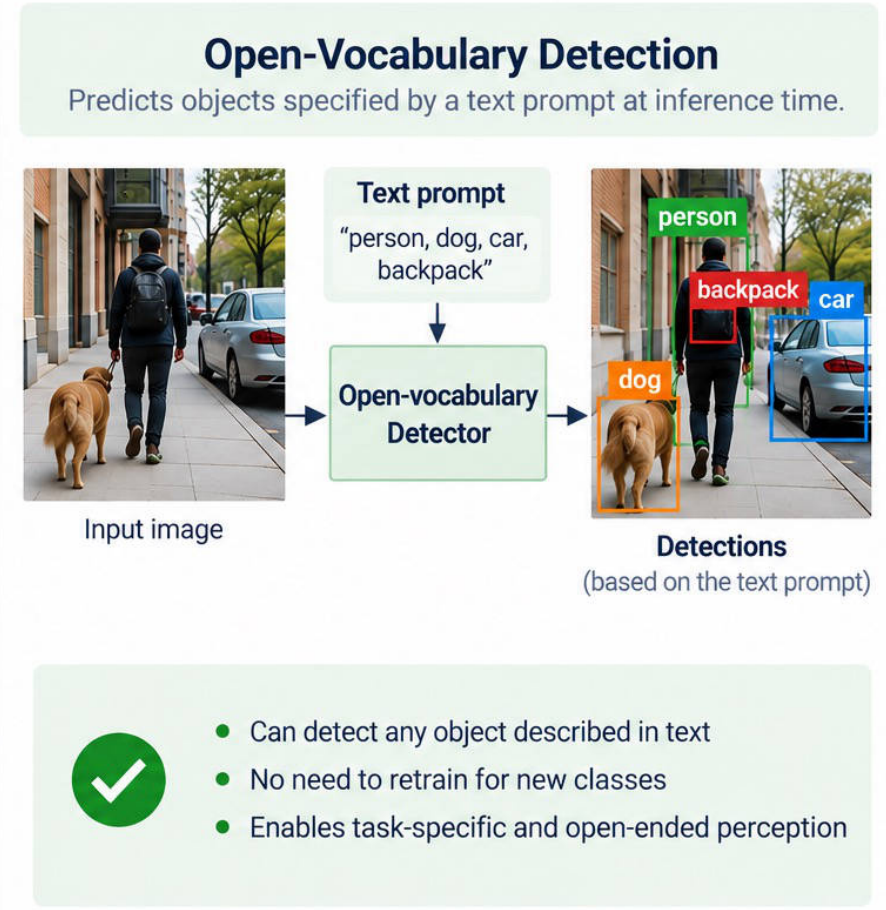
Closed Set, Open Set, Open Vocabulary

Closed Set	Open Set	Open Vocabulary
All classes that may appear at test time are known during training.	*Unknown classes may appear at test time.*	*Unknown classes may appear at test time.*
Detector: Choose among fixed known classes	Detector: Detect known classes and identify novel objects as **unknown**	Detector: detect objects using text descriptions **beyond a fixed list** of trained classes.

CRICOS No.00213J

Open-Vocabulary Detection

- Detect objects described by arbitrary text prompts, rather than choosing only from a fixed set of trained classes.
- → Not limited to a closed set of training classes



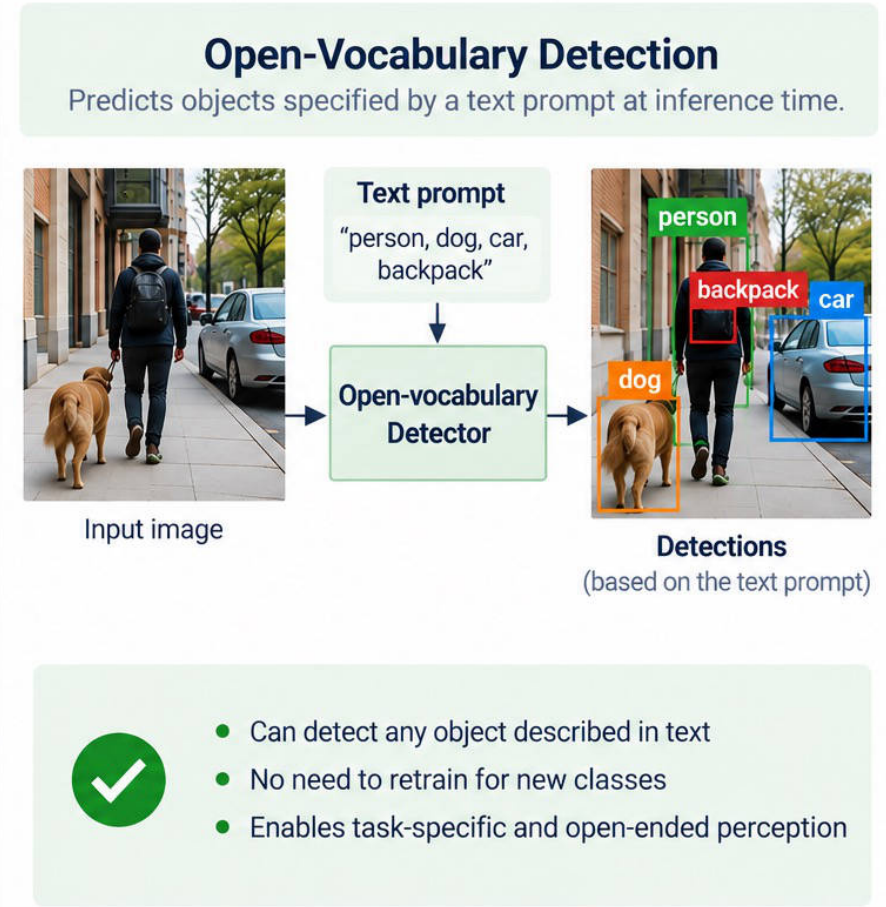
CRICOS No.00213J

Illustration by GPT-5.6

Open-Vocabulary Detection

- Detect objects described by arbitrary text prompts, rather than choosing only from a fixed set of trained classes.
- → Not limited to a closed set of training classes

But how to do it?

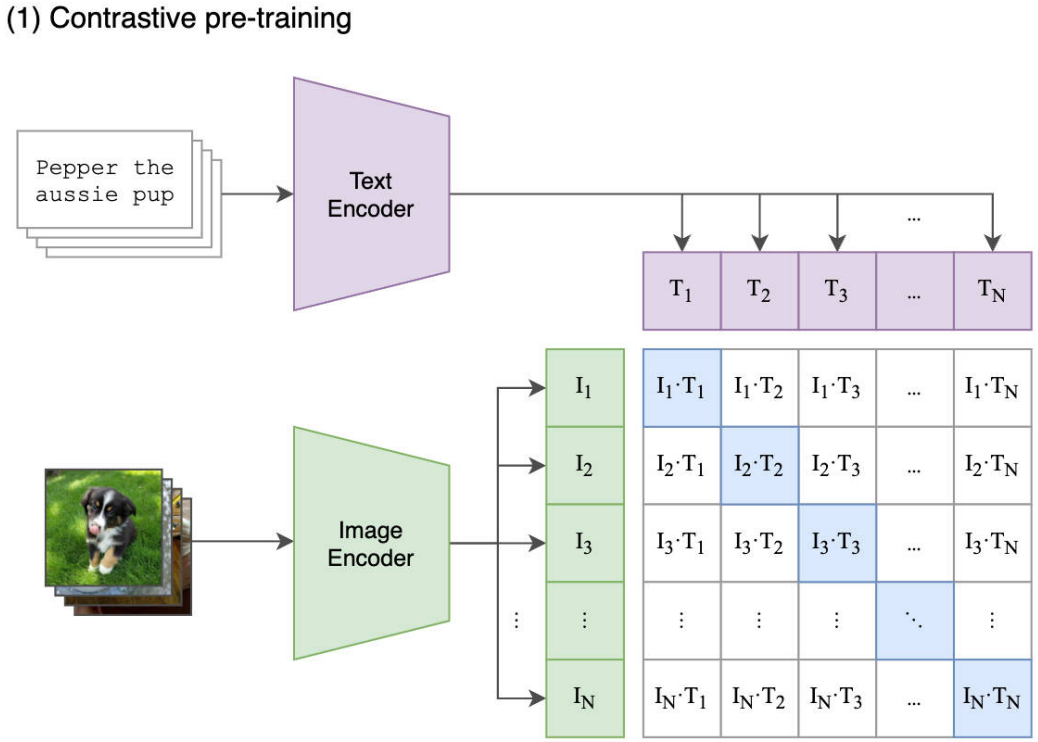


CRICOS No.00213J

Illustration by GPT-5.6

From CLIP to Object Localisation

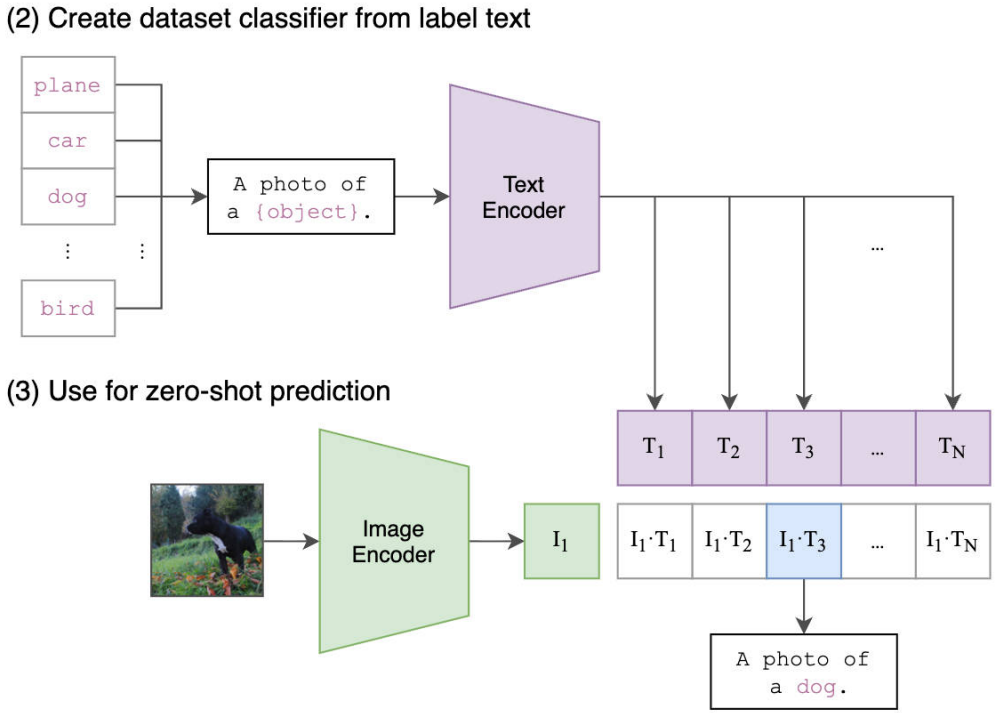
Remember CLIP from Week 8?



Idea: Train text encoder and image encoder so that features share the same space

From CLIP to Object Localisation

Remember CLIP from Week 8?

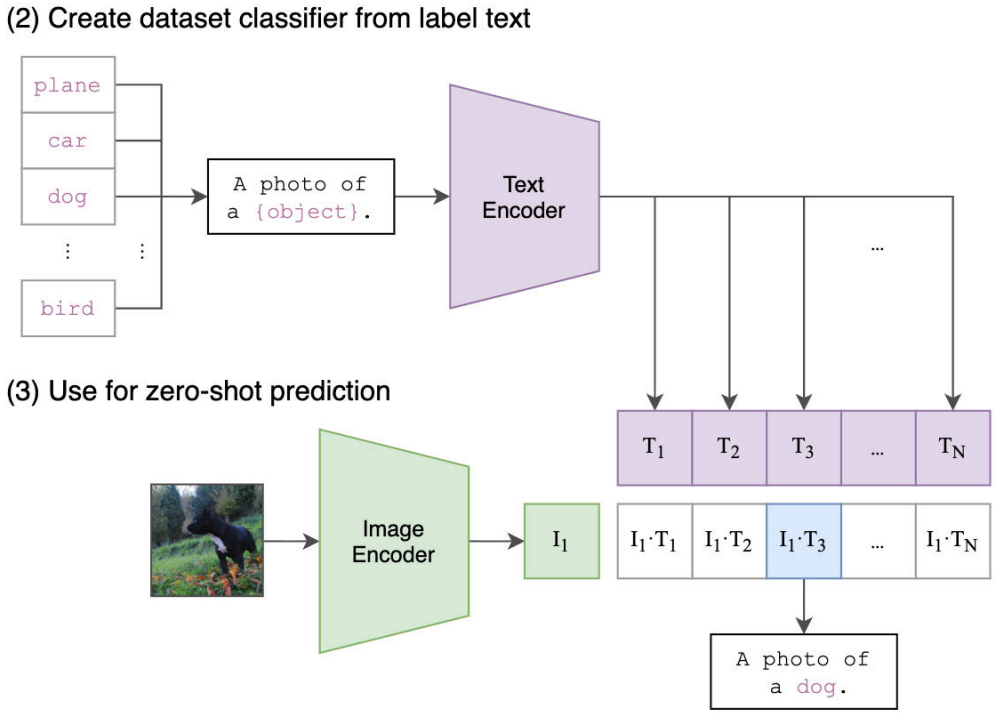


- Image → embedding vector $\boldsymbol{z}_{\mathrm{Img}}$
- Text → embedding vector $\boldsymbol{z}_{\mathrm{Text}}$
- Similarity between $\boldsymbol{z}_{\mathrm{Img}}$ and $\boldsymbol{z}_{\mathrm{Text}}$ asks*:*
  "Does this image match this concept?"

CRICOS No.00213J

From CLIP to Object Localisation

Remember CLIP from Week 8?

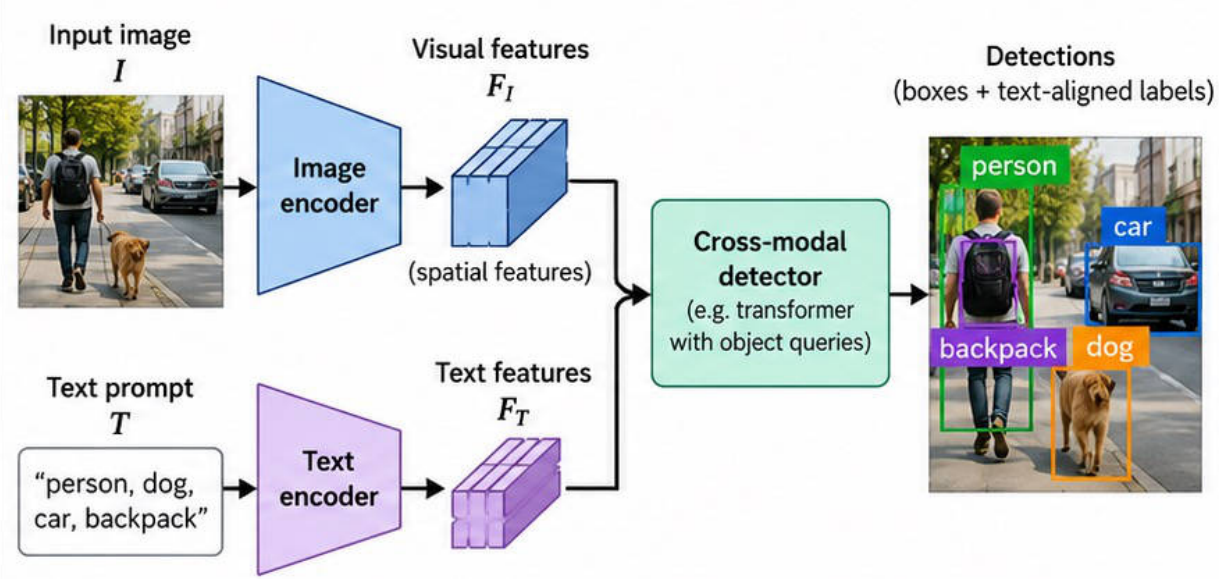


- Image → embedding vector $\boldsymbol{z}_{\mathrm{Img}}$
- Text → embedding vector $\boldsymbol{z}_{\mathrm{Text}}$
- Similarity between $\boldsymbol{z}_{\mathrm{Img}}$ and $\boldsymbol{z}_{\mathrm{Text}}$ asks: *"Does this image match this concept?"*

- *But for detection we need spatial localisation: "Where in the image is this concept?"*

CRICOS No.00213J

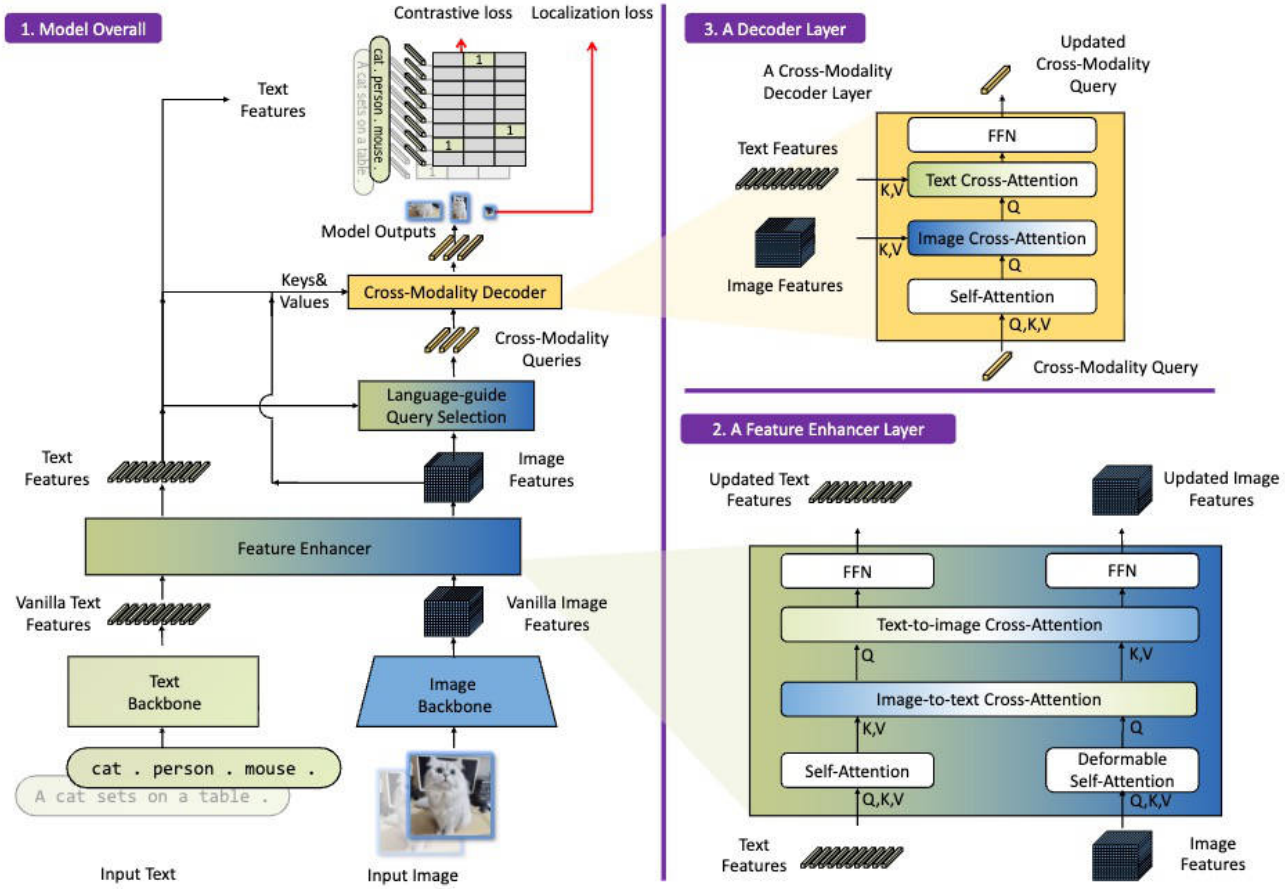
Closed-Set vs Open-Vocab Detection



- Use a detector to propose detections, and then score detections against text concepts
- Use image encoder and text encoder with aligned feature spaces
- The prediction head is now conditioned on language!

CRICOS No.00213J

Grounding DINO





Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection

Shilong Liu[1,2*], Zhaoyang Zeng[2], Tianhe Ren[2], Feng Li[2,3], Hao Zhang[2,3], Jie Yang[2,4], Qing Jiang[2,6] Chunyuan Li[5], Jianwei Yang[5], Hang Su[1], Jun Zhu[1**], Lei Zhang[2**].

[1] Dept. of Comp. Sci. and Tech., BNRist Center, State Key Lab for Intell. Tech. & Sys., Institute for AI, Tsinghua-Bosch Joint Center for ML, Tsinghua University
[2] International Digital Economy Academy (IDEA)
[3] The Hong Kong University of Science and Technology
[4] The Chinese University of Hong Kong (Shenzhen) [5] Microsoft Research, Redmond
[6] South China University of Technology
liusl20@mails.tsinghua.edu.cn, leizhang@idea.edu.cn

arXiv:2303.05499v5 [cs.CV] 19 Jul 2024

Fig. 1: (a) Closed-set object detection requires models to detect objects of pre-defined categories. (b) We evaluate models on novel objects and standard Referring expression comprehension (REC) benchmarks for model generalizations on novel objects with attributes. (c) We present an image editing application by combining Grounding DINO and Stable Diffusion [41]. Best viewed in colors.

Abstract. In this paper, we develop an open-set object detector, called Grounding DINO, by marrying Transformer-based detector DINO with grounded pre-training, which can detect arbitrary objects with human inputs such as category names or referring expressions. The key solution of open-set object detection is introducing language to a closed-set detector for open-set concept generalization. To effectively fuse language and vision modalities, we conceptually divide a closed-set detector into three phases and propose a tight fusion solution, which includes a feature enhancer, a language-guided query selection, and a cross-modality decoder for modalities fusion. We first pre-train Grounding DINO on large-scale datasets,

* This work was done when Shilong Liu, Feng Li, Hao Zhang, Jie Yang, and Qing Jiang were interns at IDEA.
** Corresponding authors.

- Built on the DETR / DINO set-prediction paradigm
- Uses both image and text features
- Object queries produce candidate object detections
- Predictions are aligned to text phrases
- Input vocabulary can be changed at inference time

Text Prompts Matter

In open-vocabulary detection, the detector is no longer defined only by its trained weights: it is also defined by the query vocabulary supplied at deployment.

- Changing the prompt can change:
  - which objects are detected
  - how detections are labelled
  - detection confidence
- Synonyms and phrasing can matter
- More specific prompts are not always better
- Prompt design therefore becomes part of system design

CRICOS No.00213J

Open Vocabulary ≠ Open-ended Visual Reasoning

- Open Vocabulary Detectors still **struggle** with:
  - Visual difficulty (small objects, occlusions, unusual viewpoints, domain shift)
  - Language difficulty (ambiguous labels, synonyms, fine-grined categories, prompt sensitivity)

- **Attributes and state** (“the damaged bolt”, “the empty cup”, “the dirty plate”) require more than simple category recognition
- **Relations and affordances** ("the cup next to the laptop”, "the object blocking the path”, "something I can grasp”) require relational or functional reasoning

CRICOS No.00213J

Other Open-Vocabulary Detectors

- Grounding DINO: strong grounding-oriented DETR-style model
- OWL-ViT / OWLv2: clean vision-language formulation
- GLIP: grounded language-image pretraining
- YOLO-World: real-time open-vocabulary detection
- YOLOE: very recent (2025) real-time promptable/open-vocabulary detection

CRICOS No.00213J

Practical Considerations

Start Simple!

Start with a strong pretrained baseline, fine-tune as needed.
Choose architecture based on the deployment constraints.
Evaluate what matters.

- A reliable simple baseline often tells you more than a complicated model with the latest architecture that has many moving parts you don't understand.

- **Fine-tuning a pretrained model** is often the best first step
  - If the target task has limited labelled data, starting from a pretrained model such as Faster R-CNN can work very well.
  - You benefit from pretrained visual features and only need to adapt the detector to the new classes/domain.

CRICOS No.00213J

Available Pre-Trained Detectors

- **TorchVision Model Library** (simple legacy models to get started)
- **MMDetection** for massive model zoo including up-to-date models
- **Hugging Face** for some very recent models
- Detectron2 (still relevant, but less actively maintained than MMDetection)

PyTorch

Object Detection

The following object detection models are available, with or without pre-trained weights:

- Faster R-CNN
- FCOS
- RetinaNet
- SSD
- SSDlite



MMDetection

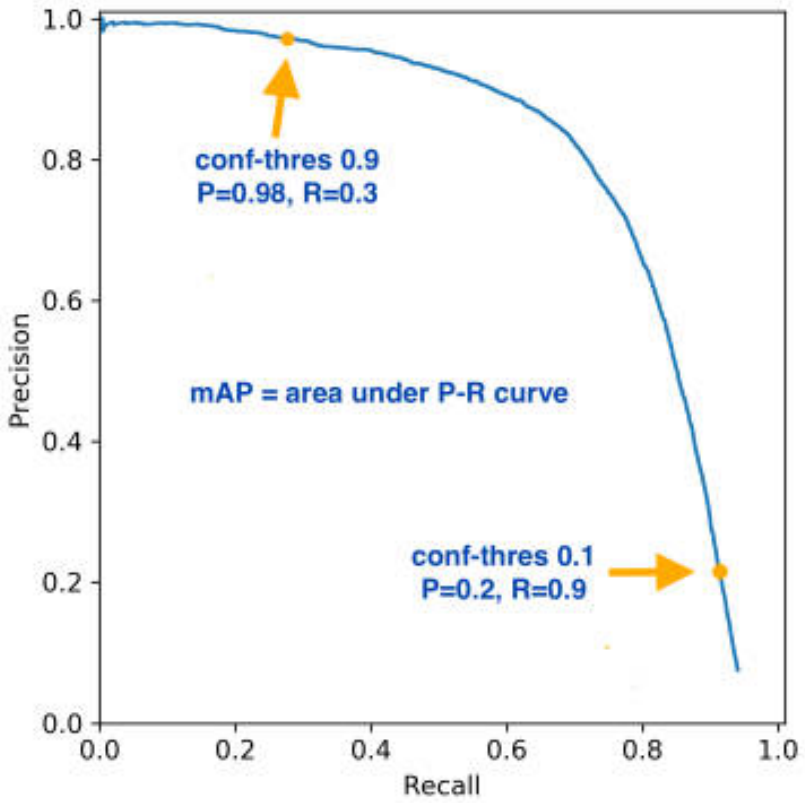
Overview of Benchmark and Model Zoo

Results and models are available in the model zoo.

Architectures

Object Detection	Instance Segmentation	Panoptic Segmentation	Other
• Fast R-CNN (ICCV'2015) • Faster R-CNN (NeurIPS'2015) • RPN (NeurIPS'2015) • SSD (ECCV'2016) • RetinaNet (ICCV'2017) • Cascade R-CNN (CVPR'2018) • YOLOv3 (ArXiv'2018) • CornerNet (ECCV'2018) • Grid R-CNN	• Mask R-CNN (ICCV'2017) • Cascade Mask R-CNN (CVPR'2018) • Mask Scoring R-CNN (CVPR'2019) • Hybrid Task Cascade (CVPR'2019) • YOLACT (ICCV'2019) • InstaBoost (ICCV'2019) • SOLO (ECCV'2020)	• Panoptic FPN (CVPR'2019) • MaskFormer (NeurIPS'2021) • Mask2Former (ArXiv'2021) • XDecoder (CVPR'2023)	• Contrastive Learning ◦ SwAV (NeurIPS'2020) ◦ MoCo (CVPR'2020) ◦ MoCov2 (ArXiv'2020) • Distillation ◦ Localization Distillation (CVPR'2022) ◦ Label Assignment Distillation (WACV'2022) • Semi-Supervised Object

Practical Considerations



Choose a confidence threshold for the type of performance that is most important for the application!

- high recall may matter more than high precision for safety-critical perception
- high precision may matter more if false detections trigger costly actions

CRICOS No.00213J

Practical Considerations

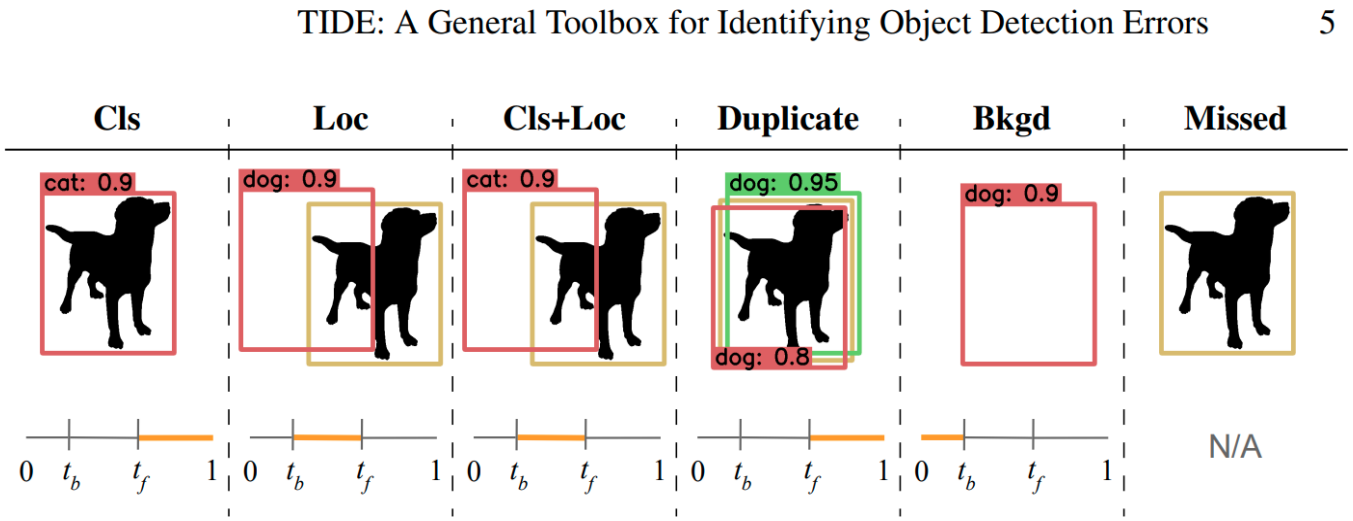


Fig. 1: **Error Type Definitions.** We define 6 error types, illustrated in the *top row*, where box colors are defined as: ■ = false positive detection; ■ = ground truth; ■ = true positive detection. The IoU with ground truth for each error type is indicated by an orange highlight and shown in the *bottom row*.

Understand what types of errors matter most for your application and assess performance for these.

Summary

Summary – Object Detection

- Families: Two-stage, single-stage, anchor-based, point-based, transformer
- Evaluation: IoU, precision & recall, AP & mAP
- Training: combine classification and localisation losses
- Open-Vocabulary: use language to define what to detect during inference

CRICOS No.00213J